

AXI4-Lite Slave: Design and Assertion-Based Verification

Complete study notes for the project, written from zero

What DV is, how every file works, what every command did, what we changed and why, the measured results, and how to build your next project alone

What this document is

These notes explain the **current** state of your project (the version tagged **final-oct4** in git) and the whole story of how it got there. They are written for someone who knows no design verification (DV) at all. Every number in the Results chapter comes from logs you pasted during our work; nothing was invented. Where something is my explanation or an assumption rather than a measured fact, the text says so.

Important honesty note about the code listings

The source listings in the Appendix were rebuilt from (1) the code you sent in your Word file and (2) every edit we made afterwards, applied in the same order. I could not read your actual files from your laptop. Two checks support them: the rebuilt testbench contains exactly 33 directed transactions (your run reported 33 scoreboard checks) and 11 lines mentioning the random generator (your own `grep -c xorshift` also gave 11). Still, small spacing differences are possible, so before the review run `cat rtl/axi4_lite_slave.sv` (and the other files) and compare with the Appendix. If something differs, trust your files.

Contents

Contents	2
How to use these notes	4
Part A. Foundations: what design verification is	5
A.1 Design versus verification	5
A.2 The words you must know	5
A.3 The DV flow	6
A.4 How a simulation works, in simple words	6
A.5 The SystemVerilog you need (with examples from your code)	7
Part B. The AXI4-Lite protocol	9
B.1 What problem a bus solves	9
B.2 Five channels, two flags each	9
B.3 The handshake: VALID and READY	9
B.4 Byte strobes (WSTRB)	10
B.5 Responses	11
B.6 Your register map and address rule	11
B.7 One real write, cycle by cycle	11
B.8 A read, in words	12
Part C. Your verification environment, file by file	13
C.1 The folder and what each file is	13
C.2 The big picture	13
C.3 The slave (axi4_lite_slave.sv)	14
C.4 The master (axi4_lite_master.sv)	17
C.5 The monitor (axi4_lite_monitor.sv)	18
C.6 The scoreboard (axi4_lite_scoreboard.sv)	18
C.7 Functional coverage (axi4_lite_coverage.sv)	20
C.8 Assertions (axi4_lite_sva.sv)	21
C.9 The testbench top (tb_top.sv)	23
C.10 Fault injection and mutants	25
Part D. Every command we used, and every script	27
D.1 Linux (WSL) commands	27
D.2 Git commands	28
D.3 Simulator commands	28
D.4 The scripts, line by line	29
Part E. History: what we did, in order, and what went wrong	32

E.1 Stage by stage	32
E.2 Problems we hit and how each was solved	33
Part F. Results: the measured numbers and what they mean	34
F.1 Clean run (final testbench, tag final-oct4)	34
F.2 Random traffic	34
F.3 RTL mutants (the slave is broken on purpose)	34
F.4 Injected master-side faults	35
F.5 What you can and cannot claim	35
F.6 Honest limitations	35
F.7 What to do next to make it stronger	36
Part G. Doing your next project alone	37
G.1 The recipe (use it on any block: FIFO, UART, I2C, arbiter ...)	37
G.2 Habits that save hours	37
G.3 Exercises (do them in this order)	37
G.4 Thirty viva questions with answers	38
Appendix. Source files (rebuilt from your code plus our edits)	40
rtl/axi4_lite_slave.sv	41
tb/master/axi4_lite_master.sv	44
tb/monitor/axi4_lite_monitor.sv	47
tb/scoreboard/axi4_lite_scoreboard.sv	49
tb/coverage/axi4_lite_coverage.sv	52
tb/assertions/axi4_lite_sva.sv	56
tb/tb_top.sv	58
scripts/build.sh	65
scripts/vbuild.sh	66
scripts/mutate.sh	67
scripts/faults.sh	68
scripts/random.sh	69
scripts/regress.sh	70

How to use these notes

The document has seven parts. You do not have to read it in one go.

Part	What it gives you	Read it when
A Foundations	What DV is, the DV flow, the SystemVerilog you need, with examples from your own code	First. Everything else builds on it.
B AXI4-Lite	The protocol: channels, signals, handshake, byte strobes, responses, a real write traced cycle by cycle	Before you open any code.
C Your environment	Every file explained: slave, master, monitor, scoreboard, coverage, assertions, fault injection, mutants, testbench top	When preparing the review and the viva.
D Commands	Every Linux, git and simulator command we used, and every script line by line	When you want to repeat or change something.
E History	What we did in order, and every error we hit with its fix	To understand why the code looks the way it does.
F Results	The measured numbers, what they mean and what they do NOT mean	Before you speak or write about results.
G Next project	A recipe, exercises and 30 viva questions with answers	When you are ready to work alone.
Appendix	Full source of every file and script	To check any line.

Reading tip

Start with Part A and B. Then open your real files next to Part C and read them side by side with the explanation. Run the commands in Part D yourself. You learn DV by running things and breaking things, not by reading.

Part A. Foundations: what design verification is

A.1 Design versus verification

A chip is described in a hardware language such as SystemVerilog. That description is the **design** (also called the RTL: register-transfer level). Before anyone spends money to build the chip, we must make sure the design does what the specification says. Finding mistakes in the design by running it in a simulator, before the chip exists, is **pre-silicon verification**. Verification is commonly said to take a large share of the effort in chip projects (often cited as about half or more; I have not checked a source for the exact figure), because bugs found after manufacturing are extremely expensive.

In simple words

Design = building a calculator. Verification = trying hard to prove the calculator is wrong: pressing every button, pressing buttons in a weird order, and checking each answer against the answers you know are right. A verification engineer's job is to **break things on purpose, safely, on a computer**.

A.2 The words you must know

Word	Meaning	In your project
DUT	Design Under Test: the thing being checked.	axi4_lite_slave.sv (instance name dut)
Testbench	All the code around the DUT that drives and checks it. Not part of the chip.	everything under tb/
Stimulus	The inputs you apply to the DUT.	the writes and reads the master performs
BFM / driver	Bus Functional Model: code that behaves like a real bus master, driving signals according to the protocol.	axi4_lite_master.sv
Monitor	Code that only WATCHES signals and reports what happened. It never drives anything.	axi4_lite_monitor.sv
Scoreboard	Keeps the expected result (a reference model) and compares it with what the DUT really did.	axi4_lite_scoreboard.sv
Assertion	A rule that is checked automatically on every clock edge, e.g. 'VALID must not drop before READY'.	axi4_lite_sva.sv (19 rules)
Coverage	A measure of WHICH situations your tests actually exercised.	axi4_lite_coverage.sv (21 bins)
Directed test	A test written by hand for one specific situation.	the basic and extra tests in tb_top.sv
Random test	Many tests with random choices (address, data, delays). A seed number makes them repeatable.	+SEED=n +RANDN=100
Regression	Running ALL tests, again and again, with one command, to see if anything broke.	scripts/regress.sh
Waveform	A picture of signals over time.	sim/waves/axi4_lite.vcd, opened in GTKWave
Mutation (mutant)	A deliberately broken copy of the design. If your testbench cannot tell it from the good one, the testbench is weak.	scripts/mutate.sh, 8 mutants
Fault injection	Deliberately corrupting a signal during simulation to see whether the checkers notice.	master fault_mode 1 to 7
Log	The text a simulation prints.	the [SCOREBOARD], [MONITOR], [SVA_FAIL] lines

A.3 The DV flow

Figure 1 shows the flow every verification project follows, whether it is a 100-line example like yours or a billion-transistor processor. The loop at the left is the most important part: you measure what you tested, find holes, write more tests, and repeat.

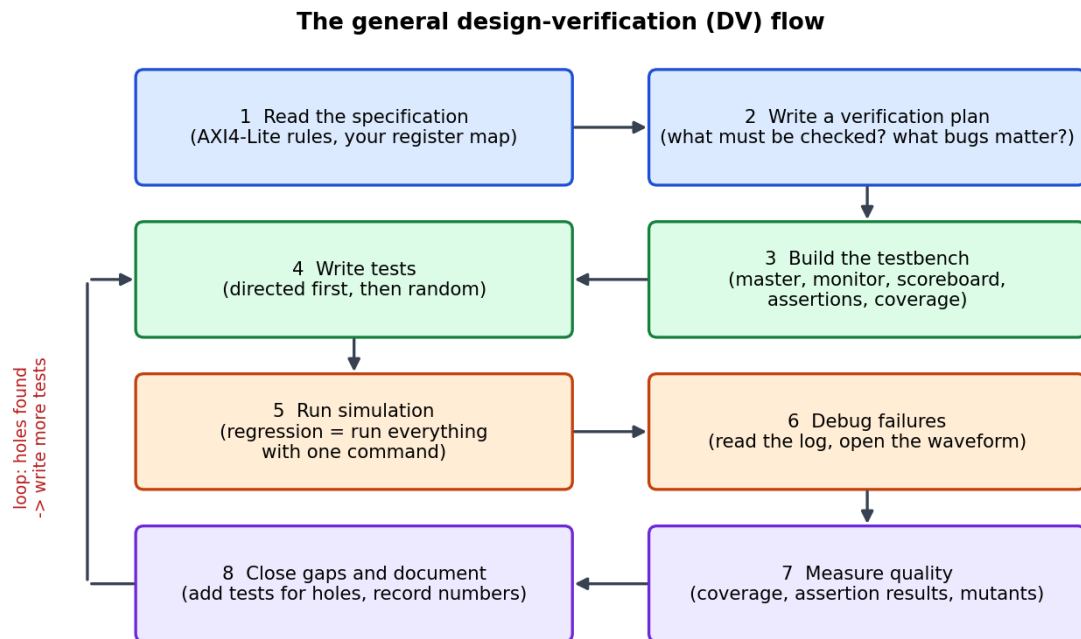


Figure 1. The general DV flow.

Here is where each step of that flow lives in your project:

Step	What it meant for you
1 Specification	The AXI4-Lite protocol rules and your own register map (four 32-bit registers at 0x0, 0x4, 0x8, 0xC).
2 Plan	What to check: data correctness, byte strobes, error responses, handshake rules, response timing.
3 Build the testbench	master, monitor, scoreboard, coverage, 19 assertions, tb_top.
4 Write tests	basic tests, 21 extra directed tests, stall and backpressure tests, unaligned tests, random traffic.
5 Run	scripts/build.sh (Icarus) and scripts/vbuild.sh (Verilator, with assertions); scripts/regress.sh runs everything.
6 Debug	reading the logs; the stray '=' error; the address bug; the Verilator deadlock; the GTKWave waveform.
7 Measure	coverage 21/21 of the defined bins; mutants killed; faults detected; random seeds passed.
8 Close gaps	the coverage run at 52.38% led to 9 new tests, which found a real bug.

A.4 How a simulation works, in simple words

A simulator is a program that pretends to be the hardware. Three phases happen:

1. **Compile**: it reads your .sv files and checks the grammar. A syntax error stops everything here (you saw this with the stray '=').
2. **Elaborate**: it connects the modules together as written in tb_top.
3. **Run**: it advances time. At every instant, whatever is scheduled to happen happens. When the testbench calls \$finish, it stops.

You used two simulators: **Icarus Verilog** (commands `iverilog` then `vvp`) and **Verilator**. Why two? Icarus 12.0 cannot run SystemVerilog assertions. Verilator 5.020 can run a useful subset of them, so the assertions run only in the Verilator build. Both run the same testbench and, after our fixes, give identical results.

A.5 The SystemVerilog you need (with examples from your code)

You are learning SystemVerilog on Udemy. This section is a map of the pieces that appear in your project, so each course topic has a place to land.

Modules and ports

A **module** is a box with named ports (wires going in and out). Your slave is one module. A module is used by writing an **instance** of it inside another module:

from `rtl/axi4_lite_slave.sv`

```
module axi4_lite_slave #(
    parameter ADDR_WIDTH = 32,
    parameter DATA_WIDTH = 32
)(
    input logic                aclk,
    input logic                aresetn,
```

- **parameter**: a constant you can set when you create an instance (here the bus widths, both 32).
- **input logic [DATA_WIDTH-1:0] x**: a 32-bit input called x. [31:0] means 32 wires, bit 31 is the top bit.
- **logic**: the general-purpose 4-state signal type of SystemVerilog (0, 1, X unknown, Z floating). wire is used for things driven by a module output (as in `tb_top`).

Clocked logic and the two kinds of assignment

Hardware that remembers things (registers) changes only on a clock edge. In code: `always_ff @(posedge aclk)` means 'do this at every rising edge of aclk'. Inside it you see `<=` (**non-blocking** assignment). In testbench code you see `=` (**blocking** assignment).

Why `<=` and `=` both exist

Think of a classroom photo. With `<=`, every student decides their new pose using what they saw at the moment of the click, and all change together after the click: this is how flip-flops behave. With `=`, each statement takes effect immediately, one after another, like ordinary programming. Rule of thumb: **`<=` for registers in clocked RTL, `=` for testbench variables and for combinational logic.**

initial, always, assign, task, function

Construct	Meaning	Example in your project
<code>initial begin ... end</code>	Runs once at time 0. Used for testbench sequences, clock and reset generation.	clock generation in <code>tb_top</code>
<code>always @(posedge clk)</code>	Runs at every rising edge, forever.	scoreboard, monitor, coverage, SVA counters
<code>always_ff</code>	Like <code>always @(posedge)</code> but tells the tool this must be a register.	the slave's main block
<code>assign y = ...</code>	Continuous connection: y always equals the expression (combinational).	<code>s_axi_awready = ...</code>
<code>task</code>	A reusable procedure that can WAIT for clock edges.	<code>master.write()</code> , <code>master.read()</code>
<code>function</code>	A reusable calculation that cannot wait.	<code>idx_of()</code> in the scoreboard, <code>xorshift()</code> in <code>tb_top</code>
<code>automatic</code>	Each call gets its own variables (needed for reusable tasks).	<code>task automatic write(...)</code>
<code>final begin ... end</code>	Runs once when the simulation ends.	the summaries printed by scoreboard, coverage and SVA

Operators and numbers you will see

Item	Meaning
32'h0000_000C	a 32-bit number written in hexadecimal; the underscores are only for readability
2'b10	a 2-bit number in binary
x[5:2]	bits 5 down to 2 of x (a 'part select')
x[8*b +: 8]	8 bits starting at bit 8*b going upward (an 'indexed part select'): byte number b
&& !	logical and, or, not
~x x ^ y	bitwise invert; bitwise exclusive-or
==	'case equality': compares including X and Z, so an unknown value never passes by accident
'0	all bits zero, whatever the width
cond ? a : b	if cond then a else b (used in the address decode)

System tasks, macros and plusargs

- `$display("text %h", x)` prints text to the log (%h hex, %0d decimal, %b binary, %0t time).
- `$time` is the current simulation time. In your logs it prints in picoseconds, so 1445000 means 1445 ns.
- `$finish` ends the simulation. `$random` and `$value$plusargs` read random numbers and command-line options like `+SEED=3`.
- ``define NAME ...` creates a text macro (the CHK macro in `tb_top`). ``ifdef ASSERTIONS ... `endif` includes code only if the build defines `ASSERTIONS` (the Verilator build does, Icarus does not).
- `dut.ready_delay = 4;` is a **hierarchical reference**: reaching into a lower module by its instance name to change a variable. Testbenches do this a lot; real hardware cannot.

Part B. The AXI4-Lite protocol

B.1 What problem a bus solves

In a chip, many blocks must talk: a processor, memories, timers, a UART. A **bus protocol** is an agreed set of wires and rules so any block can talk to any other. **AXI** (Advanced eXtensible Interface) is part of ARM's AMBA family of protocols. **AXI4-Lite** is the small version: single transfers only (no bursts), used mainly for reading and writing control and status registers. Your slave is a tiny register block with an AXI4-Lite face.

Master = the side that starts a transfer. **Slave** = the side that answers. Your testbench master plays the processor; the DUT is the slave.

B.2 Five channels, two flags each

AXI4-Lite uses five independent one-way channels. Each channel has its own VALID and READY wires plus its payload.

Channel	Direction	Payload signals	Purpose
AW write address	master to slave	AWADDR, AWVALID, AWREADY	which register to write
W write data	master to slave	WDATA,WSTRB,WVALID,WREADY	the data and which bytes to write
B write response	slave to master	BRESP,BVALID,BREADY	did the write work?
AR read address	master to slave	ARADDR,ARVALID,ARREADY	which register to read
R read data	slave to master	RDATA,RRESP,RVALID,RREADY	the value, and did the read work?

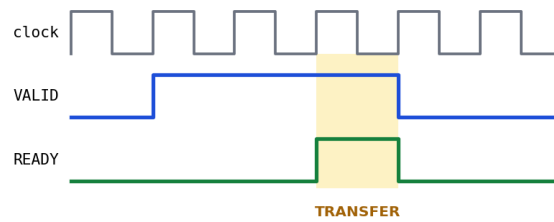
Note: as far as I know, the AXI4-Lite signal list also includes AWPROT and ARPROT (protection-type signals); confirm in the AMBA spec. Your slave does not implement them; this is listed as a limitation in Part F.

B.3 The handshake: VALID and READY

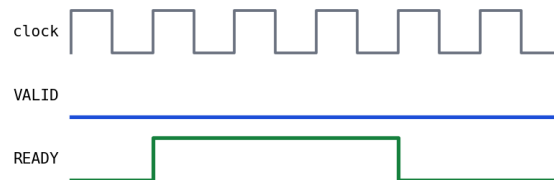
The sender of a channel raises **VALID** ('I have something'). The receiver raises **READY** ('I can take it'). The transfer happens in a clock cycle in which **both are 1**, sampled at the rising clock edge.

The VALID / READY handshake

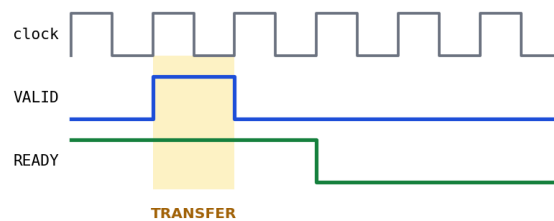
Case 1: VALID is up, READY is not ready yet -> sender WAITS, then transfer



Case 2: READY is up, but VALID is low (nothing to send) -> NO transfer



Case 3: VALID and READY both up in the same cycle -> transfer immediately



A transfer happens only in a cycle where VALID = 1 and READY = 1 (sampled at the rising clock edge).

Figure 2. Three handshake situations.

Two rules matter for your assertions. Once a sender has raised VALID, it must **keep VALID high and keep the payload unchanged until the transfer happens**. Otherwise the receiver could take half of a changing value. Rules like 'VALID must not drop' and 'the address must stay stable while waiting' are exactly what assertions A01 to A12 check. (The wording of the official rules is in the ARM AMBA AXI specification; use that document as the final authority for the viva.)

B.4 Byte strobes (WSTRB)

A 32-bit word is four bytes.WSTRB has four bits, one per byte; bit 0 is the lowest byte. A 1 means 'write this byte', a 0 means 'leave it alone'.

Register before	WDATA	WSTRB	Register after	Why
FFFFFFFF	11223344	1111	11223344	all four bytes written
FFFFFFFF	11223344	0001	FFFFFF44	only the lowest byte
FFFFFFFF	11223344	0010	FFFF33FF	only byte 1 (0x33 replaces 0xFF)
11223344	AABBCCDD	0101	11BB33DD	bytes 0 and 2 only (matches test 'WSTRB 0101 partial')
anything	anything	0000	unchanged	no byte selected: nothing changes

Check against the testbench

In `tb_top` the test starts with `FFFFFFFF`, then writes `11223344` with strobe `0001`, and expects `FFFFFF44`. Then it writes `11223344` again with `0010` and expects `FFFF3344` (byte 1 becomes 33, byte 0 stays 44 from before). Each write changes one more byte, so after four writes the register holds `11223344`. The testbench's expected values are in the Appendix (`tb_top.sv`).

B.5 Responses

BRESP / RRESP	Name	In your design
00	OKAY	valid address
01	EXOKAY	not supported in AXI4-Lite (as far as I know; confirm in the AMBA spec). Your assertions A13/A14 treat 01 as illegal.
10	SLVERR	slave error: used for every invalid address
11	DECERR	decode error: legal in AXI but your design never produces it; assertions A13/A14 also flag it as 'not legal for this design'

B.6 Your register map and address rule

Address	Register	Access
0x00000000	REG0	read / write, byte strobes honoured
0x00000004	REG1	same
0x00000008	REG2	same
0x0000000C	REG3	same
anything else	none	SLVERR; writes change nothing; reads return 0

An address is valid only if it is exactly 0x0, 0x4, 0x8 or 0xC. That includes unaligned addresses such as 0x1 or 0x2, which return SLVERR (a decision we made in the 'extensions' stage; the real AXI spec would allow other behaviour, so state it as a design choice).

B.7 One real write, cycle by cycle

The figure below was drawn from the debug print you ran on Icarus. Each column shows the value each signal had at the rising edge named above it.

One real write (REG0 = 0x12345678), values from your Icarus debug print

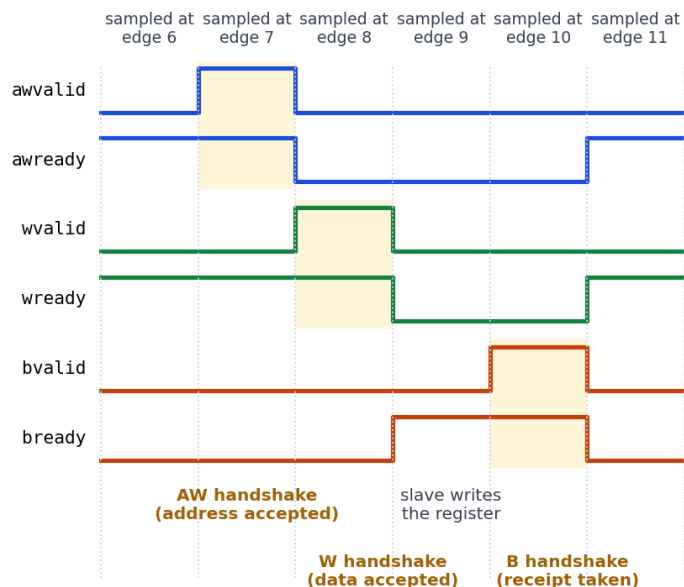


Figure 3. A write of 0x12345678 to REG0, drawn from your measured values.

Edge	What happened
7	awvalid = 1 and awready = 1: the address is accepted. From now the slave's aw_received flag is 1, so awready falls.
8	wvalid = 1 and wready = 1: the data is accepted. w_received becomes 1, so wready falls.

Edge	What happened
9	Both flags are 1 and bvalid is 0: the slave performs the write into the register and sets bvalid (visible after this edge). The master has raised bready.
10	bvalid = 1 and bready = 1: the receipt is taken. The slave clears bvalid.
11	Back to idle: awready and wready are 1 again.

B.8 A read, in words

The master puts the address on AR with ARVALID. When ARREADY is also 1, the slave looks up the register and prepares RDATA and RRESP, then raises RVALID. The master has RREADY high (or raises it after its own delay); when both are 1, the data is delivered and the slave clears RVALID. Unlike a write, a read needs only two channels.

Part C. Your verification environment, file by file

C.1 The folder and what each file is

Project layout (as of tag final-oct4)

```
axi_lite/
  rtl/
    axi4_lite_slave.sv          the DUT (the design being verified)
  tb/
    master/axi4_lite_master.sv  bus master model (BFM): drives the bus
    monitor/axi4_lite_monitor.sv watches the bus and prints handshakes
    scoreboard/axi4_lite_scoreboard.sv reference model + checker, PASS/FAIL
    coverage/axi4_lite_coverage.sv counts which scenarios were exercised (21 bins)
    assertions/axi4_lite_sva.sv 19 assertion rules (Verilator build only)
    interface/axi4_lite_if.sv   an old interface file; NOT used by any build
    tb_top.sv                   clock, reset, wiring, all tests, watchdog
    legacy/axi4_lite_fault_injection.sv old passive module, no longer used
  scripts/
    build.sh                    compile + run with Icarus (no assertions)
    vbuild.sh                   compile + run with Verilator (with assertions)
    mutate.sh                   one RTL mutant      mutate_all.sh (first version, superseded)
    faults.sh                   7 master-side faults random.sh      10 random seeds
    regress.sh                  everything, writes sim/results/regression_summary.txt
    run.sh                      your original script (not used by the new flow)
  sim/
    README.md/                 build products (ignored by git) and sim/results/ (kept in git)
    .gitignore                  this is a FOLDER (that is why the project text is in README_PROJECT.md)
                              tells git to ignore compiled files, waveforms, logs
```

C.2 The big picture

Your verification environment

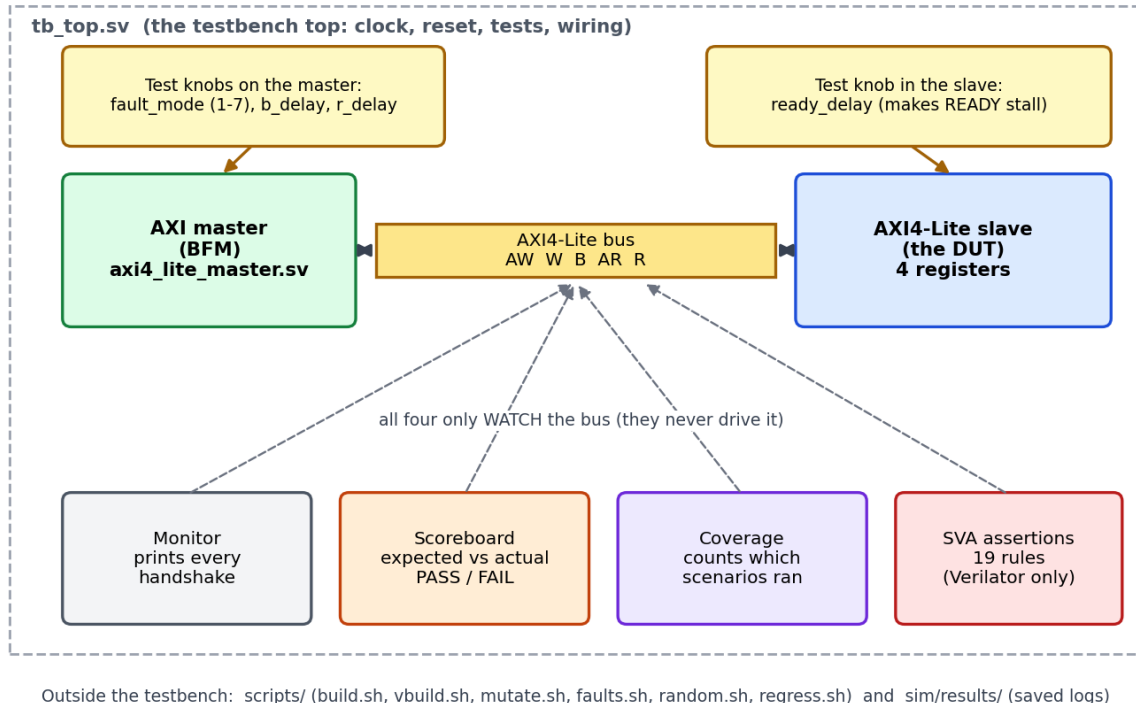


Figure 4. Your verification environment.

Read the picture from the middle outwards. The **master** and the **slave** talk over the AXI4-Lite bus. Four **observers** (monitor, scoreboard, coverage, assertions) are wired to the same bus signals, but only as inputs: they never drive anything, so they cannot disturb the test. The testbench top **tb_top** creates the clock and reset, instantiates everything, and runs the tests by calling the master's tasks. The yellow boxes are **test knobs**: variables that tests set to create stalls or inject faults.

Why observers are separate from the master

If the master also checked its own work, one wrong idea in the master would also be in the check, and the bug would hide. That is why the scoreboard has its own address map instead of copying the RTL's decode, and why the assertions are written from the protocol rules, not from the slave's code.

C.3 The slave (axi4_lite_slave.sv)

This is the design under test: four 32-bit registers behind an AXI4-Lite interface. Below, the file is explained in the order the code appears.

Ports

The ports follow the five channels. Names start with `s_axi_` ('slave AXI'). Inputs: `clk`, `aresetn` (active-low reset: 0 means reset), the master-driven signals `awaddr/awvalid`, `wdata/wstrb/wvalid`, `bready`, `araddr/arvalid`, `rready`. Outputs: `awready`, `wready`, `bresp/bvalid`, `arready`, `rdata/rresp/rvalid`.

Storage and flags

registers and the 'inbox' for a write

```
logic [DATA_WIDTH-1:0] reg0;
logic [DATA_WIDTH-1:0] reg1;
logic [DATA_WIDTH-1:0] reg2;
logic [DATA_WIDTH-1:0] reg3;
// Write Channel Internal Registers
logic [ADDR_WIDTH-1:0] awaddr_reg;
logic [DATA_WIDTH-1:0] wdata_reg;
logic [DATA_WIDTH/8-1:0] wstrb_reg;
logic aw_received;
logic w_received;
```

- **reg0..reg3**: the four registers.
- **awaddr_reg, wdata_reg, wstrb_reg**: a small inbox that holds the address, data and strobe of a write until it is executed.
- **aw_received, w_received**: flags meaning 'the address has arrived' and 'the data has arrived'. They are independent, because AXI lets the two come in either order.

The READY outputs

ready logic (continuous assignments)

```
// Write Address Ready
// Accept address only when previous address
// has already been consumed.
assign s_axi_awready = !aw_received && !s_axi_bvalid && (aw_wait >= ready_delay);
// Write Data Ready
assign s_axi_wready = !w_received && !s_axi_bvalid && (w_wait >= ready_delay);
// Read Address Ready
// Only accept a new read when previous read
// response has been consumed.
assign s_axi_arready = !s_axi_rvalid && (ar_wait >= ready_delay);
```

- **awready = !aw_received && !bvalid && (aw_wait >= ready_delay)**. In words: accept a new address only when the previous one is already stored, no old response is still waiting, and (testing only) we have stalled long enough. With `ready_delay = 0` the last part is always true.
- **wready** is the same idea for data.
- **arready = !rvalid && ...**: accept a new read only when the previous read answer has been taken.

Reset

When `aresetn` is 0 at a clock edge, every register, flag and valid output goes back to 0 (responses go back to OKAY). Assertion A15 checks that after reset is asserted, BVALID and RVALID are low.

Capturing the address and the data

two independent captures

```
// Capture Write Address
if (s_axi_awvalid && s_axi_awready) begin
    awaddr_reg <= s_axi_awaddr;
    aw_received <= 1'b1;
end
// Capture Write Data
if (s_axi_wvalid && s_axi_wready) begin
    wdata_reg <= s_axi_wdata;
    wstrb_reg <= s_axi_wstrb;
    w_received <= 1'b1;
end
```

At each clock edge, if the master is offering an address (awvalid) and the slave is ready (awready), the address is copied into the inbox and the flag is set. The same happens separately for data. Because they are separate ifs, the slave works whether the address or the data arrives first, or both together.

Executing the write

the write executes only when BOTH have arrived (first register case shown; cases 1, 2 and 3 are identical in form)

```
// Perform Write
//
// AXI allows AW and W to arrive independently.
// Therefore we wait until BOTH are received.
if (aw_received && w_received && !s_axi_bvalid) begin
    case ((awaddr_reg[31:4] == 28'd0 && awaddr_reg[1:0] == 2'b00) ? awaddr_reg[5:2] : 4'hF)
        4'd0: begin
            if (wstrb_reg[0])
                reg0[7:0] <= wdata_reg[7:0];
            if (wstrb_reg[1])
                reg0[15:8] <= wdata_reg[15:8];
            if (wstrb_reg[2])
                reg0[23:16] <= wdata_reg[23:16];
            if (wstrb_reg[3])
                reg0[31:24] <= wdata_reg[31:24];
            s_axi_bresp <= RESP_OKAY;
        end
    end
```

The address decode is the odd-looking expression after case:

```
case ((awaddr_reg[31:4] == 28'd0 && awaddr_reg[1:0] == 2'b00) ? awaddr_reg[5:2] : 4'hF)
```

- `awaddr_reg[31:4] == 28'd0`: all the upper address bits are 0, so the address is inside the small 0x00..0x0F window.
- `awaddr_reg[1:0] == 2'b00`: the address is word aligned (a multiple of 4).
- If both are true, the case selector is `awaddr_reg[5:2]`, which is 0, 1, 2 or 3 for addresses 0x0, 0x4, 0x8, 0xC. Otherwise the selector is 4'hF, which matches no register, so execution falls into default (SLVERR).
- History: the original code used only `awaddr_reg[5:2]`. That made 0x10000000 and 0x40 look like register 0. Test 'INVALID WRITE' found it (Part E).

Inside each register case, four `if (wstrb_reg[n])` lines copy only the selected bytes. After the case, `bvalid` is set to 1 and both flags are cleared, ready for the next write.

response taken

```
// Write Response Handshake
if (s_axi_bvalid && s_axi_bready) begin
    s_axi_bvalid <= 1'b0;
end
```

The read path

start of the read case

```
// READ TRANSACTION
if (s_axi_arvalid && s_axi_arready) begin
  case ((s_axi_araddr[31:4] == 28'd0 && s_axi_araddr[1:0] == 2'b00) ? s_axi_araddr[5:2] :
    + 4'hF)
    4'd0: begin
      s_axi_rdata <= reg0;
      s_axi_rresp <= RESP_OKAY;
    end
  end
```

The register is copied into rdata at the moment the address is accepted. For an invalid address rdata is 0 and rresp is SLVERR. Then rvalid is raised; it is cleared when the master takes the data (rvalid and ready both 1).

The test knob

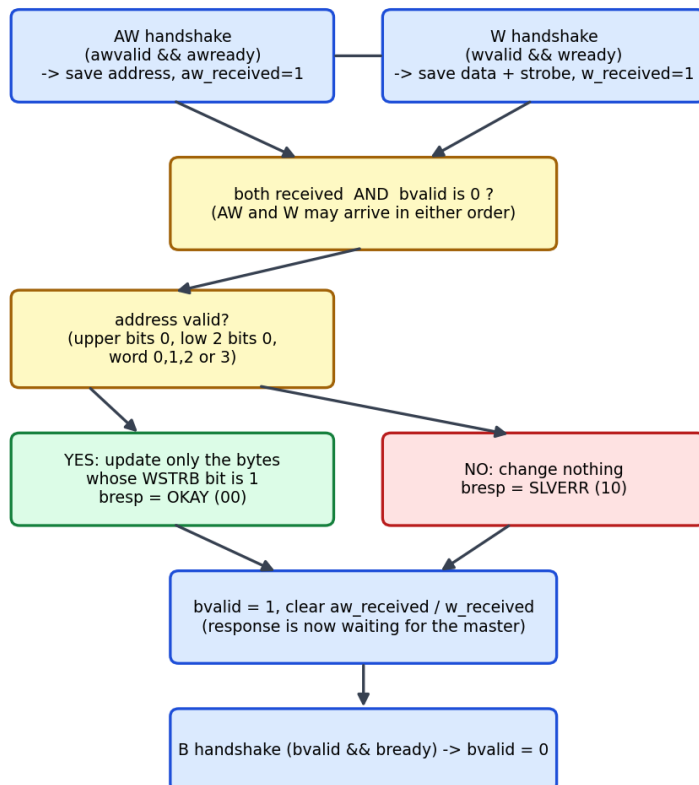
ready_delay and its counters

```
// Simulation-only knob: holds READY low for N cycles after VALID (creates stalls)
integer ready_delay = 0;
integer aw_wait = 0, w_wait = 0, ar_wait = 0;
always @(posedge aclk) begin
  aw_wait <= (s_axi_awvalid && !s_axi_awready) ? aw_wait + 1 : 0;
  w_wait <= (s_axi_wvalid && !s_axi_wready) ? w_wait + 1 : 0;
  ar_wait <= (s_axi_arvalid && !s_axi_arready) ? ar_wait + 1 : 0;
end
```

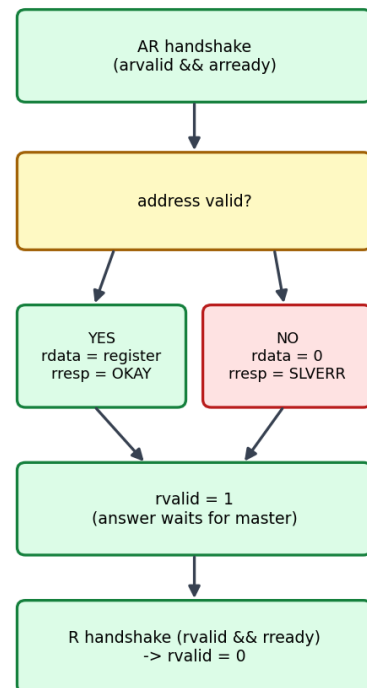
ready_delay is an integer the testbench can set (default 0). When it is 4, the slave keeps each READY low for four cycles after VALID appears, which creates the 'stall' situations that the stability assertions need. It is not synthesizable hardware; it exists only for simulation. In a future clean-up it should be wrapped in an `ifdef SIM.

What the slave does for a write and for a read

WRITE path (inside the slave)



READ path



awready / wready / arready go LOW while the slave is busy; ready_delay can hold them low longer (test knob, default 0).

Figure 5. What the slave does, step by step.

Is this an FSM?

The project documents call the slave 'FSM-based'. Honestly, the code has no enum of states. Its behaviour is controlled by flags (`aw_received`, `w_received`, `bvalid`, `rvalid`), which together form an **implicit** state machine: idle, address-only, data-only, both-received, response-pending. If asked in the viva, say exactly that. Do not claim an explicit FSM.

What this slave does NOT do (limitations)

- Only **one outstanding transaction** at a time (it will not accept a new write until the previous response is taken).
- No AWPROT / ARPROT ports.
- Invalid addresses (including unaligned ones) return SLVERR; DECERR is never used.
- WSTRB with all zeros is accepted and changes nothing (response OKAY for a valid address).
- The `ready_delay` knob is simulation-only.

C.4 The master (`axi4_lite_master.sv`)

The master is a **bus functional model**: it knows how to perform a protocol-correct write or read when a test calls `master.write(addr, data, strobe)` or `master.read(addr, data)`. It is a testbench model, not hardware.

The timing rule: act at the falling edge

The falling-edge rule used by the master

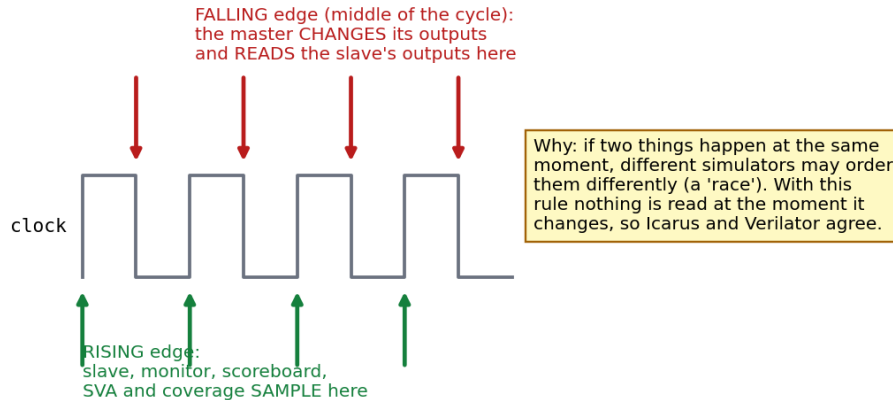


Figure 6. The master changes and samples signals in the middle of the cycle.

This rule was added after a real problem. The first master changed its outputs at the same moment the slave read them. Icarus happened to order that moment one way and Verilator the other; under Verilator the master never noticed that the slave had accepted the address, and the simulation hung (Part E, problem 7). Acting at the falling edge removes the ambiguity because the slave and all checkers only look at the rising edge.

How the write task works

address channel (simplified: the fault hooks are left out of this excerpt)

```

// Write address channel
@(negedge aclk);
awaddr = addr;
awvalid = 1'b1;
seen = awready;
@(posedge aclk);
while (!seen) begin
    @(negedge aclk);
    seen = awready;
    // (fault-injection hooks omitted here; see Appendix)
    @(posedge aclk);
end
@(negedge aclk);
awvalid = 1'b0;

```

Line by line: wait for a falling edge; drive the address and raise VALID; read READY (variable seen); wait for the next rising edge. If READY was already 1, the handshake happens at that rising edge and the loop is skipped. If not, the while (!seen) loop waits one more cycle and looks again. After the handshake, at the next falling edge VALID is dropped. The data channel and the response channel follow the same pattern. The write task is therefore strictly sequential: address, then data, then response. (This means the slave's ability to take data before the address has never been tested. See Part F, limitations.)

response channel: optional delay, then take the receipt

```

// Write response channel
repeat (b_delay) @(negedge aclk);
bready = 1'b1;
seen = bvalid;
resp_seen = bresp;
@(posedge aclk);
while (!seen) begin
    @(negedge aclk);
    seen = bvalid;
    resp_seen = bresp;
    @(posedge aclk);
end

```

- repeat (b_delay) @(negedge aclk); waits b_delay cycles before raising BREADY. With b_delay = 3 the slave must hold its response steady for 3 cycles (backpressure). The read task has the same idea with r_delay.
- last_bresp and last_rresp store the most recent response code so the tests in tb_top can check them (for example that an invalid write returned SLVERR).
- fault_mode (0 = none) selects one injected fault; see C.10.

Why task automatic? It gives each call its own copy of the local variables (seen, resp_seen). Without it, two overlapping calls could overwrite each other's variables.

C.5 The monitor (axi4_lite_monitor.sv)

The monitor is the simplest block: at every rising edge it checks the five handshakes (valid and ready both 1) and prints one line each. It also remembers the last write's address, data and strobe so it can print a summary when the write finishes. Sample lines from your logs:

lines from your run output

```

[MONITOR] AW HANDSHAKE: ADDR=00000000
[MONITOR] READ COMPLETE: DATA=cafebabe RRESP=00

```

The monitor does no checking. It is your first debugging tool: when a test fails, the monitor lines show what the bus really did.

C.6 The scoreboard (axi4_lite_scoreboard.sv)

The scoreboard is the answer key. It does its own simple version of the slave's job and compares.

How the scoreboard decides PASS or FAIL

Runs at every rising clock edge (after reset is released):

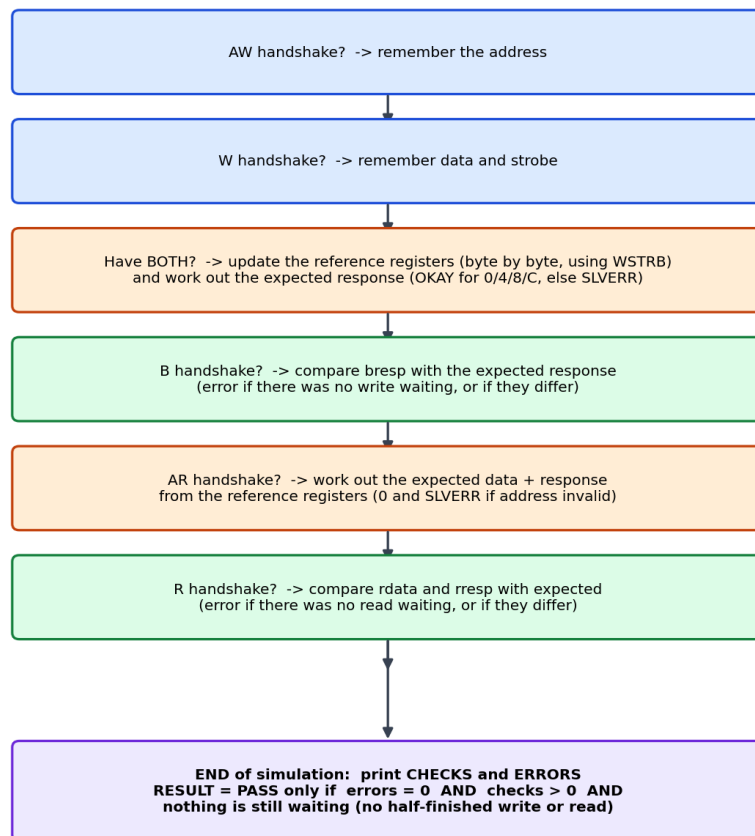


Figure 7. How the scoreboard works at every rising edge.

its own address map

```

function automatic integer idx_of(input logic [ADDR_WIDTH-1:0] a);
  case (a)
    32'h0000_0000: idx_of = 0;
    32'h0000_0004: idx_of = 1;
    32'h0000_0008: idx_of = 2;
    32'h0000_000C: idx_of = 3;
    default:      idx_of = -1;
  endcase
endfunction
  
```

idx_of returns 0..3 for the four valid addresses and -1 for anything else. It is written from the specification, deliberately **not** copied from the RTL decode. That makes it strict: an unaligned address such as 0x1 is invalid for the scoreboard, and the RTL now agrees.

updating the reference registers

```

// When both have arrived, update the model and the expected response
if (aw_got && w_got) begin
  wi = idx_of(cap_awaddr);
  if (wi >= 0) begin
    tmp = ref_reg[wi];
    for (bb = 0; bb < 4; bb = bb + 1)
      if (cap_wstrb[bb])
        tmp[8*bb +: 8] = cap_wdata[8*bb +: 8];
    ref_reg[wi] = tmp;
    exp_bresp = RESP_OKAY;
  end
  else begin
    exp_bresp = RESP_SLVERR;
  end
  bresp_pending = 1'b1;
  aw_got = 1'b0;
  w_got = 1'b0;
end
  
```

tmp[8*bb +: 8] = cap_wdata[8*bb +: 8] copies byte number bb. The loop runs for the four bytes and only copies those whose strobe bit is 1, just like the RTL but written independently.

the end-of-run verdict

```
final begin
  $display("");
  $display("=====");
  $display("                SCOREBOARD SUMMARY");
  $display("=====");
  $display("CHECKS = %0d  ERRORS = %0d", checks, errors);
  if (errors == 0 && checks > 0 && !bresp_pending && !rd_pending && !aw_got && !w_got)
    $display("SCOREBOARD RESULT: PASS");
  else
    $display("SCOREBOARD RESULT: FAIL");
end
```

Why the verdict has four conditions

A run only passes if there were no errors, at least one check happened (a testbench that checked nothing must not 'pass'), and nothing is left half-finished. The last condition was added after the mutant BVALID_DROP showed that the earlier scoreboard said PASS on a slave that hung forever.

C.7 Functional coverage (axi4_lite_coverage.sv)

Coverage answers: **what did my tests actually exercise?** A test suite can pass 100% of its tests and still miss important situations. Your coverage module keeps 21 counters called **bins**. Each counter goes up when the matching event happens on the bus. At the end, the number of non-zero counters is divided by 21.

Coverage point	Bins	Count
Write address	REG0, REG1, REG2, REG3, INVALID	5
Read address	REG0, REG1, REG2, REG3, INVALID	5
Write strobe	1111, 0001, 0010, 0100, 1000, partial, 0000	7
Write response	OKAY, SLVERR	2
Read response	OKAY, SLVERR	2
Total		21

how a bin is counted (sampled at the address handshake)

```
// WRITE ADDRESS
if (awvalid && awready) begin
  case ((awaddr[31:4] == 28'd0 && awaddr[1:0] == 2'b00) ? awaddr[5:2] : 4'hF)
    4'd0:
      write_reg0_count = write_reg0_count + 1;
    4'd1:
      write_reg1_count = write_reg1_count + 1;
    4'd2:
      write_reg2_count = write_reg2_count + 1;
    4'd3:
      write_reg3_count = write_reg3_count + 1;
    default:
      write_invalid_count =
        write_invalid_count + 1;
  endcase
end
```

- The first run printed **11 / 21 = 52.38%**: the basic tests only used valid addresses, strobe 1111 and OKAY responses.
- After 9 extra tests it printed 17/21 (80.95%), then 21/21 once invalid addresses were rejected correctly.
- **Honest reading**: '100%' means 'all 21 bins I defined were hit'. It does not mean the design is fully verified. The bins are simple single events, with no crosses (for example strobe x address).
- The coverage module uses the same address decode as the RTL, so a mistake shared by both would not show up in coverage. The scoreboard, which has its own map, is the independent check.

- Why manual counters and not SystemVerilog covergroup? Icarus 12.0 and Verilator 5.020 both rejected covergroups when you tried (Verilator printed 'Unsupported: covergroup'). With Xcelium or Questa you could use real covergroups.

A note on the 52.8% number

Some early documents said '52.8%'. Your own log said 52.38%, which is exactly 11/21. Use 52.38% (or 52.4%).

C.8 Assertions (axi4_lite_sva.sv)

An **assertion** is a rule written once and then checked automatically on every clock edge for the whole simulation. If a rule is broken, the tool reports it at that exact time. This is more reliable than looking at waveforms by eye. SystemVerilog Assertions (SVA) is the part of the language for this.

How to read one assertion

rule A01

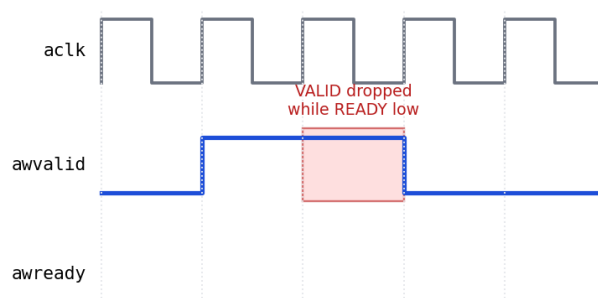
```
A01_AWVALID_HOLD: assert property (@(posedge aclk)
    (aresetn && awvalid && !awready) |>= (!aresetn || awvalid))
    else `SVA_FAIL("A01_AWVALID_HOLD")
```

Piece	Meaning
A01_AWVALID_HOLD:	a name (label) so the report says which rule failed
assert property (...)	check this property continuously
@(posedge aclk)	evaluate it at every rising clock edge
(aresetn && awvalid && !awready)	the condition on the left: 'the master offers an address and the slave is not ready, and we are out of reset'
>=	'then, on the NEXT clock edge, the right side must be true' (non-overlapping implication)
(!aresetn awvalid)	the right side: awvalid is still 1 (or reset happened, which cancels the rule)
else `SVA_FAIL("...")	what to do on failure: print a tagged line with the time and count the failure

Two operators to remember: $\rightarrow$ means 'if the left side is true now, the right side must be true now'; $\Rightarrow$ means 'if the left side is true now, the right side must be true at the next edge'. \$stable(x) means 'x has the same value as at the previous edge'.

How one assertion catches a violation

Rule A01: (awvalid && !awready) $\Rightarrow$ awvalid ('if stalled now, VALID must still be up next cycle')



rising edge #2: awvalid=1 and awready=0 -> rule A01 'wakes up'
it now demands awvalid is STILL 1 at edge #3

edge #3: awvalid is 0 -> [SVA_FAIL] A01_AWVALID_HOLD

Figure 8. How rule A01 catches a dropped VALID.

The 19 rules

Rule	In plain words	How it is built
A01 AWVALID_HOLD	address VALID must not drop while READY is low	(awvalid && !awready) => awvalid
A02 AWADDR_STABLE	address must not change while waiting	... => \$stable(awaddr)
A03 WVALID_HOLD	data VALID must not drop while waiting	(wvalid && !wready) => wvalid
A04 WDATA_STABLE	write data must not change while waiting	... => \$stable(wdata)
A05 WSTRB_STABLE	write strobe must not change while waiting	... => \$stable(wstrb)
A06 ARVALID_HOLD	read-address VALID must not drop while waiting	(arvalid && !arready) => arvalid
A07 ARADDR_STABLE	read address must not change while waiting	... => \$stable(araddr)
A08 BVALID_HOLD	the slave must hold BVALID until BREADY	(bvalid && !bready) => bvalid
A09 BRESP_STABLE	the write response must not change while waiting	... => \$stable(bresp)
A10 RVALID_HOLD	the slave must hold RVALID until RREADY	(rvalid && !rready) => rvalid
A11 RDATA_STABLE	read data must not change while waiting	... => \$stable(rdata)
A12 RRESP_STABLE	read response must not change while waiting	... => \$stable(rresp)
A13 BRESP_LEGAL	BRESP is only OKAY (00) or SLVERR (10)	bvalid -> (bresp == 00 bresp == 10)
A14 RRESP_LEGAL	RRESP is only OKAY or SLVERR	rvalid -> (rresp == 00 rresp == 10)
A15 RESET_RESP_LOW	after reset is asserted, BVALID and RVALID are low	!aresetn => (!bvalid && !rvalid)
A16 NO_B_WITHOUT_WRITE	no write response before both address and data were accepted	bvalid -> (aw_cnt > b_cnt && w_cnt > b_cnt)
A17 NO_R_WITHOUT_READ	no read data before a read address was accepted	rvalid -> (ar_cnt > r_cnt)
A18 B_TIMEOUT	a write response must arrive within 16 cycles	counter + immediate assert
A19 R_TIMEOUT	a read response must arrive within 16 cycles	counter + immediate assert

The helper counters and the timeouts

helper counters

```

wire b_pending = (aw_cnt > b_cnt) && (w_cnt > b_cnt);
wire r_pending = (ar_cnt > r_cnt);
always @(posedge aclk) begin
    if (!aresetn) begin
        aw_cnt <= 0; w_cnt <= 0; b_cnt <= 0;
        ar_cnt <= 0; r_cnt <= 0;
        b_wait <= 0; r_wait <= 0;
    end
    else begin
        if (awvalid && awready) aw_cnt <= aw_cnt + 1;
        if (wvalid && wready) w_cnt <= w_cnt + 1;
        if (bvalid && bready) b_cnt <= b_cnt + 1;
        if (arvalid && arready) ar_cnt <= ar_cnt + 1;
        if (rvalid && rready) r_cnt <= r_cnt + 1;
        if (b_pending && !bvalid) b_wait <= b_wait + 1; else b_wait <= 0;
        if (r_pending && !rvalid) r_wait <= r_wait + 1; else r_wait <= 0;
    end
end

```

The counters aw_cnt, w_cnt, b_cnt, ar_cnt, r_cnt count handshakes. b_pending is true when a write has been fully accepted but its response has not been taken. b_wait counts the cycles that a response has been pending without BVALID. The timeout rules use assert (b_wait != B_MAX_WAIT + 1): using *not equal to 17* means the failure is reported exactly once, at the moment the counter reaches 17, instead of once per cycle forever.

Why not write the timeout as ##[1:16] in real SVA?

The natural SVA way is a range delay (a response within 1 to 16 cycles). Verilator 5.020 rejected that syntax ('Unsupported: ## range cycle delay'), so the timeout is built with a counter. On Xcelium or Questa you could use the range version. In the report, say the assertions run on 'the subset of SVA supported by Verilator 5.020'.

Sanity counters: is a passing assertion meaningful?

An assertion like A01 can never fail if its left side never happens. The module therefore counts how often each trigger occurred (stall cycles on AW, W, AR, B, R). In the first assertion run all stall counts were 0, so 'SVA FAILS = 0' proved little. After the stall tests were added they became AW 4, W 4, AR 4, B 2, R 3, which shows the rules were really exercised.

Why the assertions are inside `ifdef ASSERTIONS

Icarus cannot run these assertions, so the instance in tb_top is wrapped in `ifdef ASSERTIONS. The Verilator script defines it (-DASSERTIONS), the Icarus script does not.

C.9 The testbench top (tb_top.sv)

tb_top has no ports. It is the 'universe' for the simulation. In order, it contains:

1. parameters and the AXI wires shared by everyone;
2. **clock**: forever #5 aclk = ~aclk; toggles every 5 ns, so the period is 10 ns (100 MHz);
3. **reset**: aresetn is 0 for 5 rising edges, then 1, and the log prints 'RESET RELEASED';
4. instances of the DUT (named dut), master, monitor, scoreboard, coverage, and (under `ifdef) the SVA module;
5. the CHK macro and counters, the **test sequence**, the waveform dump, and the watchdog.

the CHK macro: one check with a name

```
`define CHK(EXP, ACT, NAME) \  
  if ((ACT) === (EXP)) begin \  
    $display("[TEST] %s PASS", NAME); \  
    ext_pass = ext_pass + 1; \  
  end else begin \  
    $display("[TEST] %s FAIL EXPECTED=%h ACTUAL=%h", NAME, (EXP), (ACT)); \  
    ext_fail = ext_fail + 1; \  
  end
```

`CHK(expected, actual, "name") prints '[TEST] name PASS' or '[TEST] name FAIL EXPECTED=... ACTUAL=...' and counts it. The final line '[TEST] EXTRA TESTS: 21 passed, 0 failed' comes from those counts.

The tests, in order

Group	What it does	Why
Basic tests (4)	write then read back REG0..REG3 with distinct values	the original sanity test
WSTRB tests (6)	from FFFFFFFF write with strobes 0001, 0010, 0100, 1000, 0101, 0000 and read back	exercises every byte lane; fills 6 coverage bins
Invalid address tests (3 data checks)	write to 0x10000000 and 0x40, read from 0x10000000; check the register is unchanged / data is zero	fills the invalid bins; found the aliasing bug
Response-code checks (4)	valid write returns OKAY; the two invalid writes and the invalid read return SLVERR	checks the codes, not just the data
Backpressure test	master delays BREADY and RREADY by 3 cycles	slave must hold its response steady (rules A08-A12)
Stall test	slave holds READY low for 4 cycles on AW, W, AR	exercises rules A01-A07
Unaligned tests (4 checks)	write to 0x1, read from 0x2	unaligned = invalid = SLVERR
Random traffic (optional)	+RANDN=n random transactions; seed with +SEED=s	finds what hand-written tests missed
Fault injection (optional)	+FAULT=n runs one injected fault on top	see C.10

Counting: 8 transactions in the basic tests plus 25 in the extra tests make **33 directed transactions**, which is why a clean run reports **CHECKS = 33**. Running with +RANDN=100 adds 100, giving **CHECKS = 133**, as in your ten random runs.

Random traffic and the random generator

our own random generator

```
// Simple xorshift32 random generator (same result in every simulator)
reg [31:0] rng;
function automatic [31:0] xorshift(input [31:0] s);
    reg [31:0] x;
    begin
        x = s;
        x = x ^ (x << 13);
        x = x ^ (x >> 17);
        x = x ^ (x << 5);
        xorshift = x;
    end
endfunction
```

xorshift is a tiny, well-known pseudo-random generator: it scrambles a 32-bit number with shifts and exclusive-ors. We wrote our own because Verilator's \$random(seed) gave almost the same address pattern for different seeds (95 of 133 accesses went to one address). With xorshift, the same seed gives the same sequence in every simulator, and different seeds give different traffic: the address mix looked even and differed between seeds.

start of the random loop

```
// --- constrained-random traffic (run with +SEED=n +RANDN=count)
if (!$value$plusargs("SEED=%d", rseed)) rseed = 1;
if (!$value$plusargs("RANDN=%d", rnd_cnt)) rnd_cnt = 0;
rng = rseed ^ 32'hA5A5_1234;
for (rnd_n = 0; rnd_n < 8; rnd_n = rnd_n + 1) rng = xorshift(rng);
for (rnd_n = 0; rnd_n < rnd_cnt; rnd_n = rnd_n + 1) begin
    rng = xorshift(rng); dut.ready_delay = rng[17:16];
    rng = xorshift(rng); master.b_delay = rng[17:16];
    rng = xorshift(rng); master.r_delay = rng[17:16];
    rng = xorshift(rng); rnd_sel = rng[18:16];
    case (rnd_sel)
        0: rnd_addr = 32'h0000_0000;
        1: rnd_addr = 32'h0000_0004;
        2: rnd_addr = 32'h0000_0008;
        3: rnd_addr = 32'h0000_000C;
        4: rnd_addr = 32'h0000_0010;
        5: rnd_addr = 32'h0000_0001;
        6: rnd_addr = 32'h1000_0004;
        default: rnd_addr = 32'h0000_0040;
    endcase
    rng = xorshift(rng); rnd_data = rng;
    rng = xorshift(rng); rnd_strb = rng[19:16];
    rng = xorshift(rng);
    if (rng[16]) master.write(rnd_addr, rnd_data, rnd_strb);
    else master.read(rnd_addr, read_data);
end
dut.ready_delay = 0; master.b_delay = 0; master.r_delay = 0;
```

Each loop iteration picks random delays (ready_delay, b_delay, r_delay each 0..3), a random address from eight choices (the four registers, an invalid 0x10, an unaligned 0x1, a far address 0x10000004, and 0x40), random data and strobe, and randomly a write or a read. The scoreboard checks every one of them and the assertions watch every cycle.

Watchdog and waveform

- The **watchdog** (#500000; \$finish) ends the simulation after 500 microseconds so a hung test (for example a missing response) cannot run forever.
- \$dumpfile("sim/waves/axi4_lite.vcd"); \$dumpvars(0, tb_top); writes every signal to a VCD file that GTKWave can show. (In the Verilator build the dump is ignored because it was not built with tracing.)

C.10 Fault injection and mutants

Passing tests only prove the testbench works on a good design. To trust a checker you must show that it **fails** on a bad design. You did that in two ways.

Injected faults (the master misbehaves)

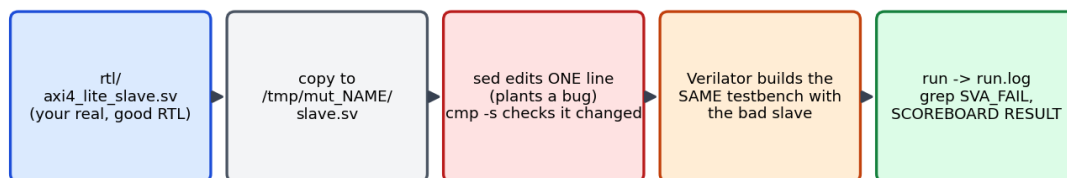
With +FAULT=n the stall test is followed by one extra transaction during which the master breaks a protocol rule once, while the slave is stalling. The expected result is that one specific assertion reports a failure.

Mode	Fault	Expected rule	Measured
1	drop AWVALID for one cycle while waiting	A01	A01 caught, 5 ns after injection
2	flip an AWADDR bit for one cycle	A02	A02 caught, 5 ns
3	drop WVALID for one cycle	A03	A03 caught, 5 ns
4	invert WDATA for one cycle	A04	A04 caught, 5 ns
5	invertWSTRB for one cycle	A05	A05 caught, 5 ns
6	drop ARVALID for one cycle	A06	A06 caught, 5 ns
7	flip an ARADDR bit for one cycle	A07	A07 caught, 5 ns

The 5 ns is half a clock cycle: the fault is created at a falling edge and the assertion checks at the next rising edge. So for these one-cycle glitches the 'detection latency' is always the same and carries little information. The scoreboard says PASS on all seven because no data was corrupted; only the assertions notice. The injection happens inside the master task (look for FAULT_INJECTED in the Appendix).

RTL mutants (the slave misbehaves)

How scripts/mutate.sh tests your testbench



Your real RTL is never modified. 'Killed' mutant = at least one checker (SVA or scoreboard) reported it.

Figure 9. How a mutant is created and judged.

Mutant	The planted bug (one sed edit)	Who should notice
MISSING_B	bvalid <= 1 becomes bvalid <= 0: no write response ever	A18 timeout
MISSING_R	rvalid <= 1 becomes rvalid <= 0: no read data ever	A19 timeout
BVALID_DROP	if (bvalid && bready) becomes if (bvalid): BVALID falls even if BREADY is low	A08
RDATA_UNSTABLE	RDATA is inverted every cycle while RREADY is low	A11
ILLEGAL_BRESP	every OKAY write response becomes 01	A13
ILLEGAL_RRESP	every OKAY read response becomes 01	A14
STRB_IGNORED	all four if (wstrb_reg[n]) become if (1'b1): strobes ignored	scoreboard / tests (not SVA)
ADDR_ALIAS	the address decode goes back to the old buggy form	scoreboard / tests (not SVA)

Result: all 8 mutants were detected by at least one checker; 6 by assertions. Two (STRB_IGNORED, ADDR_ALIAS) are invisible to assertions because the protocol is perfectly legal, only the data or address handling is wrong. That is exactly why you need both a scoreboard and assertions.

Who catches what (your measured results)

RTL mutants (bugs inside the slave) and injected master-side faults

	SVA assertions	Scoreboard / tests
MISSING_B	A18_B_TIMEOUT	caught: result FAIL
MISSING_R	A19_R_TIMEOUT	caught: result FAIL
BVALID_DROP	A08_BVALID_HOLD (+A18)	caught: result FAIL
RDATA_UNSTABLE	A11_RDATA_STABLE	caught: result FAIL
ILLEGAL_BRESP	A13_BRESP_LEGAL	caught: result FAIL
ILLEGAL_RRESP	A14_RRESP_LEGAL	caught: result FAIL
STRB_IGNORED	none	caught: result FAIL
ADDR_ALIAS	none	caught: result FAIL
fault 1 AWVALID_DROP	A01_AWVALID_HOLD	NOT caught: result PASS
fault 2 AWADDR_CHANGE	A02_AWADDR_STABLE	NOT caught: result PASS
fault 3 WVALID_DROP	A03_WVALID_HOLD	NOT caught: result PASS
fault 4 WDATA_CHANGE	A04_WDATA_STABLE	NOT caught: result PASS
fault 5WSTRB_CHANGE	A05_WSTRB_STABLE	NOT caught: result PASS
fault 6 ARVALID_DROP	A06_ARVALID_HOLD	NOT caught: result PASS
fault 7 ARADDR_CHANGE	A07_ARADDR_STABLE	NOT caught: result PASS

green = detected, red = not detected. SVA and the scoreboard catch different things: they complement each other.

Figure 10. Who caught what, from your measured regression.

Part D. Every command we used, and every script

D.1 Linux (WSL) commands

WSL (Windows Subsystem for Linux) gives you a real Linux terminal inside Windows. Your project lives in the Linux home folder `~/axi_lite`. Files under `/mnt/c/...` are your Windows drives seen from Linux; we avoided working there because it is slower and mixes line endings.

Command	What it does	Where we used it
<code>cd ~/axi_lite</code>	change directory (<code>~</code> means your Linux home)	before almost every step
<code>ls, ls -la</code>	list files; <code>-l</code> long format, <code>-a</code> include hidden files	checking that files existed
<code>cat file</code>	print a file	looking at <code>/tmp/v.log</code> , results files
<code>cat > file << 'EOF' ... EOF</code>	write the lines between the markers into a file (a 'heredoc'); quoting 'EOF' stops the shell changing <code>\$</code> and backticks	creating every script and fragment
<code>cp -r a b</code>	copy a folder with everything inside	copying the project from Windows into WSL; backups
<code>mkdir -p x</code>	make a folder (and parents), no error if it exists	<code>sim/waves</code> , <code>sim/results</code>
<code>chmod +x file</code>	allow a file to be run as a program	every script
<code>dos2unix files</code>	convert Windows line endings (CR LF) to Linux (LF)	after copying the project
<code>grep -E 'a b' file</code>	show lines matching a pattern; <code>-c</code> counts them; <code>-o</code> prints only the match; <code>-n</code> adds line numbers	everywhere
<code>sed -i 's/old/new/' file</code>	edit a file in place: replace text. Delimiter can be <code> </code> if the text contains <code>/</code>	address-decode fix, mutants, deleting a bad line (<code>sed -i '36d'</code>)
<code>awk '...' file</code>	small text-processing program: used to insert a block before a marker line and to build the mutant table	<code>tb_top.sv</code> patches, <code>regress.sh</code>
<code>head -n, tail -n</code>	first or last <code>n</code> lines	<code>tail -30</code> of the summary
<code>wc -l</code>	count lines	checking a pasted file was complete
<code>sort uniq -c</code>	count how many times each line occurs	address distribution of the random runs
<code>tee file</code>	show output AND save it to a file	<code>tee sim/results/faults.txt</code>
<code>cmd > f, cmd >> f, cmd 2>&1</code>	redirect output into a file (overwrite), append, or merge error messages into the output	logs in <code>/tmp</code>
<code>a b</code>	pipe: the output of <code>a</code> becomes the input of <code>b</code>	simulation <code>grep ...</code>
<code>cmd &</code>	run in the background so the terminal stays free	the 20-minute regression
<code>sleep n</code>	wait <code>n</code> seconds	waiting for the background run
<code>jobs, kill -9 \$(jobs -p)</code>	list background/stopped jobs; kill them all	after <code>Ctrl+Z</code> accidents
<code>timeout 30 cmd</code>	stop a command that runs longer than 30 s	protects against hung simulations
<code>stdbuf -o0 cmd</code>	make output appear immediately instead of in blocks	so killed runs still show their lines
<code>bash -n script</code>	check a script's syntax without running it	after creating each script
<code>set +H</code>	turn off history expansion so <code>!</code> is not special	fixed the 'event not found' errors
<code>md5sum</code>	fingerprint of text; equal text gives equal fingerprint	comparing seed outputs

Ctrl+C versus Ctrl+Z

Ctrl+C stops (kills) the running program. **Ctrl+Z** only *pauses* it and leaves it hanging around as a 'Stopped' job. Several times a long build looked frozen (Verilator prints nothing for 1-2 minutes), you pressed a key, and the run was cut. Rule: when a script says it takes minutes, press nothing until you see the prompt or a DONE message.

D.2 Git commands

Git keeps a history of your project so you can always go back. The mini-glossary:

Command	Meaning
git init	start tracking this folder
git add -A	stage all changes for the next commit
git commit -m "text"	save a snapshot with a message
git status --short	list changed files (nothing printed = clean)
git log --oneline head -3	last three commits
git show --stat HEAD	which files the last commit changed
git rm --cached file	stop tracking a file but keep it on disk (we did this for build outputs)
git checkout -b extensions	create and switch to a new branch (a parallel line of history)
git checkout master	switch back to the main branch
git merge extensions	bring the branch's work into the current branch (it was a 'fast-forward')
git tag name	put a permanent label on the current commit (review-oct7, final-oct4)
git checkout file	restore a file to its last committed version (undoes a mutant or a bad edit)

.gitignore lists files git should not track: `sim/*.out` (compiled program), `sim/waves/*.vcd`, `sim/logs/`, `sim/vl_obj/`. Early commits showed thousands of changed lines because these build products were being tracked; fixing `.gitignore` cured it.

D.3 Simulator commands

Icarus Verilog (scripts/build.sh)

```
iverilog -g2012 -o sim/axi4_lite.out rtl/axi4_lite_slave.sv tb/master/axi4_lite_master.sv ... tb/tb_top.sv
vvp sim/axi4_lite.out
gtkwave sim/waves/axi4_lite.vcd
```

- `iverilog` compiles. `-g2012` = use the SystemVerilog-2012 language rules. `-o` = name of the output program. The list is every source file.
- `vvp` runs the compiled program. Options after the program name, such as `+SEED=3 +RANDN=100`, are 'plusargs' that the testbench reads with `$value$plusargs`.
- `gtkwave` opens a `.vcd` waveform file.

Verilator (scripts/vbuild.sh)

```
verilator --binary --timing --assert -Wno-fatal -DASSERTIONS --top-module tb_top -Mdir sim/vl_obj -o vtb
+ <files>
./sim/vl_obj/vtb +SEED=3 +RANDN=100
```

Flag	Meaning
--binary	build a ready-to-run executable (needs no C++ wrapper from you)
--timing	support delays and event waits such as #5 and @(negedge) in the testbench
--assert	enable assertion checking
-Wno-fatal	warnings do not stop the build
-DASSERTIONS	define the macro ASSERTIONS so `ifdef ASSERTIONS code is included
--top-module tb_top	tb_top is the top of the design
-Mdir sim/vl_obj	put build files in this folder

Flag	Meaning
-o vtb	name of the executable

Verilator compiles your design into C++ and builds a program, so the first build takes one to two minutes and prints nothing while it works.

D.4 The scripts, line by line

scripts/build.sh and scripts/vbuild.sh

build.sh

```
#!/bin/bash
cd "$(dirname "$0")/.." || exit 1
iverilog -g2012 -o sim/axi4_lite.out rtl/axi4_lite_slave.sv tb/master/axi4_lite_master.sv
+ tb/monitor/axi4_lite_monitor.sv tb/scoreboard/axi4_lite_scoreboard.sv
+ tb/coverage/axi4_lite_coverage.sv tb/tb_top.sv && vvp sim/axi4_lite.out
```

- `cd "$(dirname "$0")/.." || exit 1`: go to the project folder no matter where you run the script from; stop if that fails. (\$0 is the script's own path.)
- The compile line, then `&& vvp . . .`: run only if the compile succeeded.

vbuild.sh

```
#!/bin/bash
cd "$(dirname "$0")/.." || exit 1
verilator --binary --timing --assert -Wno-fatal -DASSERTIONS --top-module tb_top -Mdir sim/vl_obj -o vtb \
rtl/axi4_lite_slave.sv tb/master/axi4_lite_master.sv tb/monitor/axi4_lite_monitor.sv \
tb/scoreboard/axi4_lite_scoreboard.sv \
tb/coverage/axi4_lite_coverage.sv tb/assertions/axi4_lite_sva.sv tb/tb_top.sv 2>&1 | grep -E "%Error" |
+ head -20
timeout 30 stdbuf -o0 ./sim/vl_obj/vtb 2>&1
```

It compiles with Verilator, shows only lines containing %Error (`grep -E "%Error" | head -20`), then runs the program with a 30-second limit.

scripts/mutate.sh NAME

mutate.sh (the mutation-flow figure in C.10 shows the idea)

```
#!/bin/bash
# Usage: scripts/mutate.sh <MUTANT_NAME>
cd "$(dirname "$0")/.." || exit 1
M=$1
D=/tmp/mut_$(M)
rm -rf "$D"; mkdir -p "$D"
cp rtl/axi4_lite_slave.sv "$D/slave.sv"
case "$M" in
  MISSING_B)
    sed -i "s/s_axi_bvalid <= 1'b1;/s_axi_bvalid <= 1'b0;/g" "$D/slave.sv" ;;
  MISSING_R)
    sed -i "s/s_axi_rvalid <= 1'b1;/s_axi_rvalid <= 1'b0;/g" "$D/slave.sv" ;;
  BVALID_DROP)
    sed -i "s/if (s_axi_bvalid && s_axi_bready) begin/if (s_axi_bvalid) begin/" "$D/slave.sv" ;;
  RDATA_UNSTABLE)
    sed -i "s/if (s_axi_rvalid && s_axi_rready) begin/if (s_axi_rvalid && !s_axi_rready) s_axi_rdata <=
+ ~s_axi_rdata;\n          if (s_axi_rvalid && s_axi_rready) begin/" "$D/slave.sv" ;;
  ILLEGAL_BRESP)
    sed -i "s/s_axi_bresp <= RESP_OKAY;/s/RESP_OKAY/2'b01/" "$D/slave.sv" ;;
  ILLEGAL_RRESP)
    sed -i "s/s_axi_rresp <= RESP_OKAY;/s/RESP_OKAY/2'b01/" "$D/slave.sv" ;;
  STRB_IGNORED)
    sed -i "s/if (wstrb_reg[[0-3]])/if (1'b1)/g" "$D/slave.sv" ;;
  ADDR_ALIASES)
    sed -i "s/case (([A-Za-z_]*)\[31:4\] == 28'd0 && \[1:0\] == 2'b00) ? \[1\][5:2\] : 4'hF)/case
+ (\[1\][5:2\])/g" "$D/slave.sv" ;;
*) echo "unknown mutant: $M"; exit 1 ;;
esac
if cmp -s rtl/axi4_lite_slave.sv "$D/slave.sv"; then
  echo "=== MUTANT: $M === MUTATION DID NOT APPLY (pattern not found)"; exit 1
fi
verilator --binary --timing --assert -Wno-fatal -DASSERTIONS --top-module tb_top -Mdir "$D/obj" -o vtb \
"$D/slave.sv" tb/master/axi4_lite_master.sv tb/monitor/axi4_lite_monitor.sv \
tb/scoreboard/axi4_lite_scoreboard.sv \
tb/coverage/axi4_lite_coverage.sv tb/assertions/axi4_lite_sva.sv tb/tb_top.sv 2>&1 | grep "%Error" | head
+ -5
timeout 120 stdbuf -o0 "$D/obj/vtb" $MUT_ARGS > "$D/run.log" 2>&1
echo "=== MUTANT: $M ==="
echo "SVA rules that fired:"
grep -o "\[SVA_FAIL\] \[A-Z0-9\]*" "$D/run.log" | sort | uniq -c | sed 's/^/ /'
first=$(grep -m1 "\[SVA_FAIL\]" "$D/run.log")
echo "First SVA failure: ${first:-none}"
grep -E "SCOREBOARD RESULT|EXTRA TESTS|WATCHDOG" "$D/run.log" | sed 's/^/ /'
```

- Copies the slave to /tmp/mut_NAME/slave.sv, applies ONE sed edit chosen by the case, and checks with `cmp -s` that the file really changed (otherwise prints MUTATION DID NOT APPLY).
- Builds and runs the unchanged testbench against the mutated slave. \$MUT_ARGS lets you pass +SEED/+RANDN for random-traffic mutant runs.
- The last lines summarise: which SVA rules fired, the first failure with its time, the test and scoreboard verdicts and whether the watchdog fired.

scripts/faults.sh, random.sh, regress.sh

faults.sh

```
#!/bin/bash
cd "$(dirname "$0")/.." || exit 1
mkdir -p sim/results
names=(x AWVALID_DROP AWADDR_CHANGE WVALID_DROP WDATA_CHANGE WSTRB_CHANGE ARVALID_DROP ARADDR_CHANGE)
for n in 1 2 3 4 5 6 7; do
  timeout 60 stdbuf -o0 sim/vl_obj/vtb +FAULT=$n > /tmp/f_$(n).log 2>&1
  inj=$(grep -m1 "FAULT_INJECTED" /tmp/f_$(n).log | grep -o "time \[0-9\]*" | cut -d' ' -f2)
  line=$(grep -m1 "\[SVA_FAIL\]" /tmp/f_$(n).log)
  rule=$(echo "$line" | awk '{print $2}')
  t=$(echo "$line" | awk '{print $5}')
  sb=$(grep "SCOREBOARD RESULT" /tmp/f_$(n).log | awk '{print $NF}')
  if [ -n "$inj" ] && [ -n "$t" ]; then lat=$(( (t - inj) / 1000 )); else lat="n/a"; fi
  echo "${names[$n]}: injected=${inj:-none} rule=${rule:-none} fail_t=${t:-none} latency_ns=$lat
+ scoreboard=${sb:-none}"
done | tee sim/results/faults.txt
```

For each fault number: run the already-built Verilator program with +FAULT=n; take the injection time from the FAULT_INJECTED line and the first [SVA_FAIL] line; compute the difference in ns with shell arithmetic `(( (t - inj) / 1000 ))` (times are in picoseconds).

random.sh

```
#!/bin/bash
cd "$(dirname "$0")/.." || exit 1
mkdir -p sim/results
for s in 1 2 3 4 5 6 7 8 9 10; do
  timeout 120 stdbuf -o0 sim/vl_obj/vtb +SEED=$s +RANDN=${RANDN:-100} > /tmp/r_$.log 2>&1
  chk=$(grep -m1 "CHECKS =" /tmp/r_$.log | sed 's/ */ /g')
  sb=$(grep "SCOREBOARD RESULT" /tmp/r_$.log | awk '{print $NF}')
  sva=$(grep "SVA FAILS" /tmp/r_$.log | awk '{print $NF}')
  tf=$(grep "EXTRA TESTS" /tmp/r_$.log | awk '{print $6}')
  echo "seed=$s $chk scoreboard=${sb:-none} sva_fails=${sva:-none} tests_failed=${tf:-none}"
done | tee sim/results/random.txt
```

Runs seeds 1 to 10 with 100 random transactions each ({RANDN: -100} means 'use \$RANDN if set, else 100') and prints one summary line per seed.

regress.sh

```
#!/bin/bash
cd "$(dirname "$0")/.." || exit 1
mkdir -p sim/results
OUT=sim/results/regression_summary.txt
: > $OUT
echo "[1/4] Clean run, Icarus" >&2
{
  echo "== CLEAN RUN (Icarus) =="
  ./scripts/build.sh | grep -E "EXTRA TESTS|CHECKS|SCOREBOARD RESULT|COVERED BINS|COVERAGE"
} >> $OUT
echo "[2/4] Clean run, Verilator with assertions" >&2
{
  echo "== CLEAN RUN (Verilator + SVA) =="
  ./scripts/vbuild.sh 2>&1 | grep -E "EXTRA TESTS|CHECKS|SCOREBOARD RESULT|SVA FAILS|SANITY"
} >> $OUT
echo "[3/4] RTL mutants (about 10-15 min, please wait)" >&2
: > sim/results/mutants.txt
for m in MISSING_B MISSING_R BVALID_DROP RDATA_UNSTABLE ILLEGAL_BRESP ILLEGAL_RRESP STRB_IGNORED
  + ADDR_ALIAS; do
  echo "$m" >&2
  ./scripts/mutate.sh $m >> sim/results/mutants.txt
done
{
  echo "== RTL MUTANTS =="
  awk '
function flush() { if (name != "") printf "%-16s SVA_caught=%-3s scoreboard=%-4s tests_failed=%s\n", name,
  + sva, sb, tf }
/=== MUTANT:/ { flush(); name=$3; sva="NO"; sb="n/a"; tf="n/a" }
/First SVA failure:/ { if ($0 !~ /none/) sva="YES" }
/SCOREBOARD RESULT/ { sb=$NF }
/EXTRA TESTS/ { tf=$6 }
END { flush() }
' sim/results/mutants.txt
} >> $OUT
echo "[4/4] Master-side faults" >&2
{
  echo "== MASTER-SIDE FAULTS =="
  ./scripts/faults.sh
} >> $OUT
cat $OUT
```

Four stages: clean Icarus run, clean Verilator run, the 8 mutants (about 10-15 minutes), and the 7 faults. Everything is appended to `sim/results/regression_summary.txt`. The `awk` block turns the mutant log into the 8-line table by reading the 'First SVA failure', 'SCOREBOARD RESULT' and 'EXTRA TESTS' lines.

About mutate_all.sh and random seeds

`mutate_all.sh` was the first way to run all mutants; `regress.sh` now does the same job inside its stage 3. `random.sh` is separate: run it after `vbuild.sh` has built the program.

Part E. History: what we did, in order, and what went wrong

What we did, in order



Figure 11. The path from the original code to the final project.

E.1 Stage by stage

Stage	What changed	Key commands / files	Measured result
Start (Windows)	Original RTL and testbench from your Word file	iverilog, vvp	4 basic tests pass; coverage 11/21 = 52.38%
WSL move	installed iverilog, gtkwave, make, dos2unix, git; copied project; removed a stray '=' on scoreboard line 36	apt install, cp -r, dos2unix, sed -i '36d'	same output as Windows
Batch 1	9 new tests (WSTRB lanes, invalid addresses); address decode changed to check upper bits	python patch of tb_top, sed on 3 files	3 of 9 failed before the fix; after: 9/9, coverage 21/21
Batch 2a/2b	master rewritten (response capture); scoreboard rewritten (own map, response checks, PASS/FAIL); first mutant demo	new master and scoreboard files	26 checks, 0 errors; mutant caught
Verilator port	master acts at the falling edge	new master; scripts/vbuild.sh	identical results on both simulators

Stage	What changed	Key commands / files	Measured result
Batch 3	19-rule assertion module; tb_top instance under ifdef	axi4_lite_sva.sv	0 SVA fails; stall counts 0 (rules not yet exercised)
Batch 4	master b_delay/r_delay; watchdog; 8 mutants; scoreboard fails on unfinished work	mutate.sh	B stall 2 and R stall 3; 6 of 8 mutants caught by SVA
Batch 5	slave ready_delay; master fault modes 1-7; stall test; fault runner	faults.sh	AW/W/AR stall 4 each; 7 of 7 faults caught
Regression	one-command run + summary file	regress.sh	summary saved in sim/results
Extensions	unaligned = invalid; random traffic with own generator; mutants under random traffic	random.sh	10 seeds x 100 transactions all PASS; 133 checks each
Final	full regression, merge to master, tags review-oct7 and final-oct4, backup	git merge, git tag	21 extra tests pass; 33 directed checks; 21/21 bins

E.2 Problems we hit and how each was solved

Debugging is most of DV work. Each row is a real event from our session; study the pattern 'symptom, then evidence, then cause, then fix'.

#	Symptom	Cause	Fix and lesson
1	iverilog: 'syntax error' at scoreboard line 36	a lone '=' character left over from pasting	printed the lines with cat -n sed -n, deleted line 36 with sed -i '36d'. Lesson: look at the exact line before editing.
2	'bash: !xxx: event not found'	the shell treats '!' as a history command inside double quotes	set +H (or avoid ! in interactive double quotes). Lesson: the code was fine; the terminal was the problem.
3	a pasted Python block gave SyntaxError	the long paste was mangled	avoided Python; used awk/sed and small files. Lesson: paste one block at a time.
4	scripts or files seemed cut off after a paste	two long heredocs pasted together interleaved in the terminal	check with bash -n and wc -l after creating a file; paste one block at a time.
5	README.md: 'Is a directory'	your project had a folder named README.md	wrote README_PROJECT.md instead.
6	git commits with thousands of changed lines	compiled files and the .vcd were tracked	git rm --cached and .gitignore.
7	Verilator hung after the first AW handshake	the master read READY at the same instant it changed; Icarus and Verilator order that moment differently	debug print compared both simulators cycle by cycle; the master now acts at the falling edge (see the falling-edge figure in C.4). Lesson: never sample and change a signal at the same instant.
8	(caught in my dry run, before you ran it) one mutant pattern would not have applied	the copy of your code I tested had hidden non-breaking spaces from the Word file, which broke a pattern anchored to the start of the line	patterns no longer anchor on indentation. Lesson: dry-run an edit script on a copy first. Whether your own files have such characters is unknown.
9	random runs gave 95 of 133 accesses to one address, same mix for different seeds	Verilator's \$random(seed) behaved differently from what we assumed	wrote our own xorshift generator; checked the address mix per seed. Lesson: verify that 'random' is really random.
10	fault latency printed as 0	integer division by the wrong unit	changed the divisor to get ns: 5 ns (half a cycle).
11	scoreboard said PASS on a slave that hung	it only checked completed transactions	final verdict now also requires nothing left pending.
12	(found by review, not by a failure) the original scoreboard and coverage used the same address decode as the RTL	a shared assumption can hide a shared mistake	the new scoreboard has its own address map written from the spec. Lesson: a checker must not copy the design's assumptions.

A positive finding: the 9 extra tests of Batch 1 failed 3 times before the decode fix, proving the testbench can find a real RTL bug, and all passed afterwards.

Part F. Results: the measured numbers and what they mean

Every number below was printed by your own runs and pasted into our chat. Nothing here is an estimate. Files that hold the raw numbers: `sim/results/regression_summary.txt`, `mutants.txt`, `faults.txt`, `random.txt`, `mutants_random_all.txt`.

F.1 Clean run (final testbench, tag final-oct4)

Item	Icarus	Verilator + assertions
Extra directed tests	21 passed, 0 failed	21 passed, 0 failed
Scoreboard checks	33 checks, 0 errors	33 checks, 0 errors
Scoreboard verdict	PASS	PASS
Functional coverage	21 / 21 bins = 100.00%	(coverage module also runs; same bins)
SVA failures	not run (Icarus cannot run them)	0
Sanity: stall cycles AW / W / AR	-	4 / 4 / 4
Sanity: stall cycles B / R	-	2 / 3
Sanity: B / R handshakes	-	16 / 17

The coverage story: 11/21 = 52.38% at the start; 17/21 = 80.95% after the 6 strobe tests; 21/21 once invalid addresses were rejected correctly.

F.2 Random traffic

Ten seeds (1 to 10), each with 100 random transactions on top of the 33 directed ones: every run reported **CHECKS = 133, ERRORS = 0, scoreboard PASS, SVA failures 0, tests failed 0**. The check count is the same for every seed because each random transaction makes exactly one check (33 + 100). After the generator fix, the address mix was even and differed between seeds, for example (accesses per address):

Address	0x0	0x1	0x2	0x4	0x8	0xC	0x10	0x40	0x1000 0000	0x1000 0004
seed 1	32	11	1	17	25	14	14	10	2	7
seed 2	31	11	1	16	14	14	11	17	2	16

These counts include the directed tests' own accesses (for example 0x1 and 0x2 come from the unaligned tests as well). Before the fix, seed 1 sent 95 of about 133 accesses to 0x8 and seeds 1 and 2 had identical mixes; that run is not used as evidence.

F.3 RTL mutants (the slave is broken on purpose)

Mutant	SVA caught it?	Rule that fired (first measured at the Batch-4 testbench)	Scoreboard	Tests failed
MISSING_B	yes	A18 at 255 ns	FAIL	n/a (hang)
MISSING_R	yes	A19 at 295 ns	FAIL	n/a (hang)
BVALID_DROP	yes	A08 at 1145 ns (and A18)	FAIL	n/a (hang)
RDATA_UNSTABLE	yes	A11 at 1195 ns (fired 3 times)	FAIL	1
ILLEGAL_BRESP	yes	A13 at 95 ns (fired 14 times)	FAIL	3
ILLEGAL_RRESP	yes	A14 at 125 ns (fired 16 times)	FAIL	0
STRB_IGNORED	NO	none	FAIL	8
ADDR_ALIAS	NO	none	FAIL	10

The times in column 3 come from the first runs of each mutant; the final testbench has more tests, so a rerun can show different times (see `sim/results/mutants.txt`). The yes/no pattern and the final regression's SVA/scoreboard/tests-failed columns are from the final regression. 'n/a (hang)' means the testbench never reached its test summary because the master waited for a response that never came; the watchdog ended the run.

Same mutants under random traffic (seed 1, 100 transactions): the same pattern. Six caught by SVA with the same first rules, STRB_IGNORED (8 tests failed) and ADDR_ALIAS (10 tests failed) caught only by the scoreboard and tests; scoreboard FAIL on all 8.

Totals: 8 of 8 mutants detected by at least one checker; 6 of 8 by assertions; 8 of 8 by the scoreboard (after it learned to fail on unfinished work). Before that fix, the scoreboard said PASS for MISSING_R and BVALID_DROP (and its FAIL on MISSING_B was an accident of a rule that fails when zero checks happened).

F.4 Injected master-side faults

Fault	Injected at	Rule	Failed at	Latency	Scoreboard
AWVALID_DROP	1440 ns	A01	1445 ns	5 ns	PASS
AWADDR_CHANGE	1440 ns	A02	1445 ns	5 ns	PASS
WVALID_DROP	1490 ns	A03	1495 ns	5 ns	PASS
WDATA_CHANGE	1490 ns	A04	1495 ns	5 ns	PASS
WSTRB_CHANGE	1490 ns	A05	1495 ns	5 ns	PASS
ARVALID_DROP	1440 ns	A06	1445 ns	5 ns	PASS
ARADDR_CHANGE	1440 ns	A07	1445 ns	5 ns	PASS

7 of 7 injected faults were caught by the intended rule. The scoreboard did not notice any of them, because no data was corrupted.

F.5 What you can and cannot claim

You CAN say	You must NOT say
A simplified AXI4-Lite slave was verified with a custom non-UVM SystemVerilog testbench.	That it is a complete or industry-grade AXI verification environment.
Coverage reached 21/21 of the 21 functional bins I defined.	'100% coverage' with no qualifier.
19 assertions ran on Verilator 5.020 with 0 false alarms on clean and random traffic.	That assertions were executed on Xcelium or Questa (they were not).
8 of 8 mutants and 7 of 7 injected faults I defined were detected; SVA and scoreboard catch complementary bug classes.	That the method detects 'all' bugs, or a general detection rate.
The design is a simplified academic slave with one outstanding transaction.	That the slave is an explicit FSM.

F.6 Honest limitations

- The mutants and faults were designed by us; 8/8 and 7/7 mean 'all that we chose', not general detection.
- The master is strictly sequential (address, then data, then response). The slave's handling of data arriving before the address, or both at once, has **never been tested**.
- One outstanding transaction only; no pipelining or out-of-order behaviour.
- Only SLVERR is used for errors (no DECERR); no AWPROT/ARPROT; unaligned addresses are invalid by our choice.
- Coverage is 21 simple bins with no crosses; the coverage module shares the RTL's decode.
- Assertions use the subset of SVA that Verilator 5.020 supports (no ## ranges, no sequences); timeouts are counter-based.
- The assertion rules and the mutant list were chosen by us, so they may share blind spots.

- No formal verification and no gate-level or timing checks.
- The ready_delay and fault knobs live in the RTL/master as simulation-only code.

F.7 What to do next to make it stronger

1. Add a master task that sends the data first, or the address and data together, and check the slave (this could reveal a real bug).
2. Add cross coverage (strobe x address, response x channel) and, if you can reach Xcelium or Questa, real covergroups and the full-SVA version of the timeouts.
3. Add AWPROT/ARPROT ports and a DECERR option; wrap ready_delay in `ifdef SIM.
4. Read the papers you collected and compare: what faults and metrics do they use that you do not?

Part G. Doing your next project alone

G.1 The recipe (use it on any block: FIFO, UART, I2C, arbiter ...)

1. **Read the spec and write the rules in plain sentences.** Each sentence is a future check.
Example: 'WSTRB bit n enables byte n'.
2. **Decide the interface and the reference behaviour.** What goes in, what must come out, and when?
3. **Build the smallest working testbench first:** clock, reset, one DUT instance, one write and one read, a waveform dump. Run it. Open the waveform.
4. **Add a driver/master task** that performs one transaction according to the protocol. Keep it falling-edge safe.
5. **Add a monitor** that prints every transaction. It will be your debugging eyes.
6. **Add a scoreboard with its own reference model.** Do not copy the RTL's logic; write it from the spec.
7. **Add directed tests** for each rule, including illegal inputs. Check results with a CHK-style macro.
8. **Add coverage counters** for the situations that matter. Run, see which bins are empty, and write tests for them.
9. **Add assertions** for protocol and timing rules. Add counters that prove each rule's trigger actually happened (non-vacuous).
10. **Add random traffic with a seed** and run many seeds. Check that different seeds really differ.
11. **Prove your checkers can fail:** create mutants of the RTL and inject faults. Every undetected one is a hole in your testbench.
12. **Automate:** one script builds, runs everything and writes a summary. Commit with git after every working step.
13. **Write down limitations honestly.**

G.2 Habits that save hours

- Change one thing, run, then commit. If it breaks, you know what broke it.
- Print before you guess: add a `$display` or open the waveform before theorising.
- Read the first error, not the last. Compile errors cascade.
- Check that a fix really applied (`grep -c`, `wc -l`, `bash -n`, `cmp -s`) before running anything long.
- Do not press keys during a long silent build.
- When two simulators disagree, compare them cycle by cycle with a small debug print; races show up as a difference.
- Never trust a green result you have not tried to make red.

G.3 Exercises (do them in this order)

1. Run `./scripts/build.sh` and `./scripts/vbuild.sh` and read every line of output. Find the line that proves 21 extra tests passed.
2. Open the VCD in GTKWave and find the AW, W and B handshakes of the first write. Match them to Figure 3.
3. Change the first basic test's data value in `tb_top`, rerun, and see where the log changes.
4. In the slave, break one thing by hand (change `RESP_SLVERR` to `RESP_OKAY` in the write default). Which checks fail? Restore with git checkout.
5. Add a new mutant to `mutate.sh` (for example make `arready` always 0) and predict which rule fires before you run it.

6. Add a new assertion: 'bvalid must never be 1 while a write address is still waiting for its ready'. Does it ever fire? Is it vacuous?
7. Write a master task `write_w_first` that sends the data before the address. Does the slave pass? (This is the biggest untested path.)
8. Add a cross-coverage counter for (strobe 0001, REG1).
9. Run `+RANDN=1000` for five seeds. Do any new failures appear?
10. Port the whole flow to a different block of your choice, writing the plan first.

G.4 Thirty viva questions with answers

#	Question	Answer
1	What is design verification?	Checking, by simulation before the chip is built, that the design meets its specification. A verification engineer tries to break the design and measures how thoroughly it was tested.
2	What is a DUT and what is a testbench?	The DUT is the design under test (here the AXI4-Lite slave). The testbench is the code that drives and checks it; it is not part of the chip.
3	What is AXI4-Lite?	The simple version of ARM's AXI bus protocol: single register-style transfers, no bursts, five channels (AW, W, B, AR, R).
4	Explain VALID and READY.	The sender raises VALID when it has data; the receiver raises READY when it can accept. A transfer happens in a cycle where both are 1, sampled at the rising clock edge. Once VALID is up, the sender must hold it and the payload steady until the transfer.
5	What isWSTRB?	A 4-bit byte-enable for the 32-bit write data: bit n set means byte n is written, otherwise it is left alone. With 0000 nothing changes.
6	What responses does your design return?	OKAY (00) for the four valid register addresses and SLVERR (10) for every other address. DECERR and EXOKAY are never produced.
7	How does your slave handle address and data arriving separately?	It captures them with independent flags (<code>aw_received</code> , <code>w_received</code>) and writes only when both are set. But my master always sends the address first, so the data-first case is untested.
8	Is the slave an FSM?	Not an explicit one: there is no enum of states. Its behaviour is controlled by flags (<code>aw_received</code> , <code>w_received</code> , <code>bvalid</code> , <code>rvalid</code>) which form an implicit state machine.
9	What is a BFM?	A bus functional model: testbench code that performs protocol-correct transactions. Mine is the master with <code>write()</code> and <code>read()</code> tasks.
10	Why does the master act on the falling edge?	To avoid a race: the slave and checkers sample at the rising edge, so the master changes and reads signals in the middle of the cycle. Without it, Icarus and Verilator ordered the same moment differently and Verilator deadlocked.
11	What does the monitor do?	It only watches the bus and prints each handshake. It never checks or drives anything.
12	What does the scoreboard check?	It keeps reference copies of the four registers, applies byte strobes, and compares every write response, read data and read response with what the slave returns. It ends with PASS only if there were no errors, at least one check, and nothing left unfinished.
13	Why does the scoreboard have its own address map?	If it copied the RTL's decode, a mistake in the decode would be copied too and never detected. It is written from the specification.
14	What is functional coverage? What does your 100% mean?	Coverage measures which scenarios the tests exercised. My 100% means all 21 bins I defined were hit (addresses, strobes, responses). It does not mean the design is fully verified; the bins are simple and have no crosses.
15	Why manual coverage counters instead of covergroups?	Icarus 12.0 and Verilator 5.020 did not support covergroups. With Xcelium or Questa I would use real covergroups.
16	What is an assertion?	A rule checked automatically at every clock edge for the whole simulation. When it is broken, the tool reports it at that exact time.
17	Explain <code> =></code> with an example.	'a <code> =></code> b' means: if a is true now, b must be true at the next clock edge. Example: (<code>awvalid && !awready</code>) <code> =></code> <code>awvalid</code> says that a stalled VALID must still be up in the next cycle.
18	Why are your assertions not run on Icarus?	Icarus 12.0 cannot execute concurrent SVA. They run on Verilator 5.020, which supports a subset; for example it rejects <code>##</code> range delays, so my timeouts use counters.

#	Question	Answer
19	What is a vacuous pass and how did you handle it?	A rule passes vacuously if its trigger never happens. I added sanity counters. In the first run all stall counts were 0, so the stability rules proved nothing; after adding backpressure and stall tests they became non-zero (AW/W/AR 4, B 2, R 3).
20	What is fault injection? Which faults did you inject?	Deliberately corrupting signals to see whether the checkers notice. Seven master-side faults: AWVALID drop, AWADDR change, WVALID drop, WDATA change, WSTRB change, ARVALID drop, ARADDR change. All seven were caught by the intended rule 5 ns (half a cycle) after injection.
21	What is a mutant and why use mutants?	A deliberately broken copy of the RTL. If the testbench cannot distinguish it from the good design, the testbench has a hole. I made eight mutants; all were detected by at least one checker.
22	Which bugs did the assertions not catch, and why?	STRB_IGNORED and ADDR_ALIAS. The bus protocol was legal; only the data or address handling was wrong. Those need the scoreboard.
23	Which bugs did the scoreboard not catch?	The seven one-cycle protocol glitches: no data was corrupted, so the scoreboard said PASS. Only assertions noticed.
24	What real bug did your testing find?	Address decode aliasing: the original code decoded only address bits [5:2], so 0x10000000 and 0x40 were treated as register 0. Nine extra tests exposed it (3 failed); the fix checks that the upper bits are zero. Unaligned addresses were made invalid later.
25	Why random testing, and what is a seed?	Random traffic finds cases hand-written tests miss. A seed makes the sequence repeatable. I ran 10 seeds of 100 transactions, all passing with 133 checks each.
26	Why did you write your own random generator?	Verilator's \$random(seed) gave nearly the same address pattern for different seeds (95 of about 133 accesses to one address). A xorshift32 generator gives the same behaviour in every simulator.
27	What is a regression?	Running all tests and checks with one command, so any change that breaks something is noticed. Mine is scripts/regress.sh.
28	What are the limitations of your work?	Simplified slave; one outstanding transaction; sequential master; data-before-address untested; 21 simple coverage bins; no DECERR or PROT signals; assertions limited to Verilator's SVA subset; faults and mutants chosen by me.
29	What would you do next?	Test data-before-address and simultaneous AW/W, add cross coverage, add PROT signals and DECERR, run the assertions on Xcelium or Questa, and compare against related work.
30	What did you learn about debugging?	Look at the exact evidence first (log lines, waveform, a cycle-by-cycle print), change one thing at a time, and check that a fix applied before running anything long.

Appendix. Source files (rebuilt from your code plus our edits)

Read this first

These listings were rebuilt, not copied from your laptop (see the note on the first page). Line numbers are for orientation only. If a listing differs from your file, your file is the truth.

Files, in order: slave, master, monitor, scoreboard, coverage, assertions, tb_top, then the scripts.

rtl/axi4_lite_slave.sv

224 lines

```
1 module axi4_lite_slave #(
2     parameter ADDR_WIDTH = 32,
3     parameter DATA_WIDTH = 32
4 )
5     input logic          aclk,
6     input logic          aresetn,
7     // AXI4-Lite Write Address
8
9     input logic [ADDR_WIDTH-1:0] s_axi_awaddr,
10    input logic               s_axi_awvalid,
11    output logic               s_axi_awready,
12    // AXI4-Lite Write Data
13
14    input logic [DATA_WIDTH-1:0] s_axi_wdata,
15    input logic [DATA_WIDTH/8-1:0] s_axi_wstrb,
16    input logic               s_axi_wvalid,
17    output logic               s_axi_wready,
18    // AXI4-Lite Write Response
19
20    output logic [1:0]          s_axi_bresp,
21    output logic               s_axi_bvalid,
22    input logic                s_axi_bready,
23
24    // AXI4-Lite Read Address
25
26    input logic [ADDR_WIDTH-1:0] s_axi_araddr,
27    input logic               s_axi_arvalid,
28    output logic               s_axi_arready,
29    // AXI4-Lite Read Data
30
31    output logic [DATA_WIDTH-1:0] s_axi_rdata,
32    output logic [1:0]          s_axi_rresp,
33    output logic               s_axi_rvalid,
34    input logic                s_axi_rready
35 );
36
37 // AXI4 Response Encoding
38
39 localparam logic [1:0] RESP_OKAY = 2'b00;
40 localparam logic [1:0] RESP_SLVERR = 2'b10;
41 // Register Bank
42 // Address:
43 //
44 // 0x00 -> REG0
45 // 0x04 -> REG1
46 // 0x08 -> REG2
47 // 0x0C -> REG3
48
49 logic [DATA_WIDTH-1:0] reg0;
50 logic [DATA_WIDTH-1:0] reg1;
51 logic [DATA_WIDTH-1:0] reg2;
52 logic [DATA_WIDTH-1:0] reg3;
53
54 // Write Channel Internal Registers
55
56 logic [ADDR_WIDTH-1:0] awaddr_reg;
57 logic [DATA_WIDTH-1:0] wdata_reg;
58 logic [DATA_WIDTH/8-1:0] wstrb_reg;
59 logic aw_received;
60 logic w_received;
61 // Simulation-only knob: holds READY low for N cycles after VALID (creates stalls)
62 integer ready_delay = 0;
63 integer aw_wait = 0, w_wait = 0, ar_wait = 0;
64 always @(posedge aclk) begin
65     aw_wait <= (s_axi_awvalid && !s_axi_awready) ? aw_wait + 1 : 0;
66     w_wait <= (s_axi_wvalid && !s_axi_wready) ? w_wait + 1 : 0;
67     ar_wait <= (s_axi_arvalid && !s_axi_arready) ? ar_wait + 1 : 0;
68 end
69 // Write Address Ready
70 // Accept address only when previous address
71 // has already been consumed.
72
```

```

73 assign s_axi_awready = !aw_received && !s_axi_bvalid && (aw_wait >= ready_delay);
74 // Write Data Ready
75
76 assign s_axi_wready = !w_received && !s_axi_bvalid && (w_wait >= ready_delay);
77
78 // Read Address Ready
79 // Only accept a new read when previous read
80 // response has been consumed.
81
82 assign s_axi_arready = !s_axi_rvalid && (ar_wait >= ready_delay);
83
84 // Sequential Logic
85
86 always_ff @(posedge aclk) begin
87     if (!aresetn) begin
88         // Reset register bank
89         reg0 <= '0;
90         reg1 <= '0;
91         reg2 <= '0;
92         reg3 <= '0;
93         // Reset write channel
94         awaddr_reg <= '0;
95         wdata_reg <= '0;
96         wstrb_reg <= '0;
97         aw_received <= 1'b0;
98         w_received <= 1'b0;
99         // Reset responses
100        s_axi_bvalid <= 1'b0;
101        s_axi_bresp <= RESP_OKAY;
102        // Reset read channel
103        s_axi_rvalid <= 1'b0;
104        s_axi_rdata <= '0;
105        s_axi_rresp <= RESP_OKAY;
106    end
107    else begin
108        // Capture Write Address
109
110        if (s_axi_awvalid && s_axi_awready) begin
111            awaddr_reg <= s_axi_awaddr;
112            aw_received <= 1'b1;
113        end
114
115        // Capture Write Data
116
117        if (s_axi_wvalid && s_axi_wready) begin
118            wdata_reg <= s_axi_wdata;
119            wstrb_reg <= s_axi_wstrb;
120            w_received <= 1'b1;
121        end
122        // Perform Write
123        //
124        // AXI allows AW and W to arrive independently.
125        // Therefore we wait until BOTH are received.
126
127        if (aw_received && w_received && !s_axi_bvalid) begin
128            case ((awaddr_reg[31:4] == 28'd0 && awaddr_reg[1:0] == 2'b00) ? awaddr_reg[5:2] :
129                + 4'hF)
130                4'd0: begin
131                    if (wstrb_reg[0])
132                        reg0[7:0] <= wdata_reg[7:0];
133                    if (wstrb_reg[1])
134                        reg0[15:8] <= wdata_reg[15:8];
135                    if (wstrb_reg[2])
136                        reg0[23:16] <= wdata_reg[23:16];
137                    if (wstrb_reg[3])
138                        reg0[31:24] <= wdata_reg[31:24];
139                    s_axi_bresp <= RESP_OKAY;
140                end
141                4'd1: begin
142                    if (wstrb_reg[0])
143                        reg1[7:0] <= wdata_reg[7:0];
144                    if (wstrb_reg[1])
145                        reg1[15:8] <= wdata_reg[15:8];

```

```

145         if (wstrb_reg[2])
146             reg1[23:16] <= wdata_reg[23:16];
147         if (wstrb_reg[3])
148             reg1[31:24] <= wdata_reg[31:24];
149         s_axi_bresp <= RESP_OKAY;
150     end
151     4'd2: begin
152         if (wstrb_reg[0])
153             reg2[7:0] <= wdata_reg[7:0];
154         if (wstrb_reg[1])
155             reg2[15:8] <= wdata_reg[15:8];
156         if (wstrb_reg[2])
157             reg2[23:16] <= wdata_reg[23:16];
158         if (wstrb_reg[3])
159             reg2[31:24] <= wdata_reg[31:24];
160         s_axi_bresp <= RESP_OKAY;
161     end
162     4'd3: begin
163         if (wstrb_reg[0])
164             reg3[7:0] <= wdata_reg[7:0];
165         if (wstrb_reg[1])
166             reg3[15:8] <= wdata_reg[15:8];
167         if (wstrb_reg[2])
168             reg3[23:16] <= wdata_reg[23:16];
169         if (wstrb_reg[3])
170             reg3[31:24] <= wdata_reg[31:24];
171         s_axi_bresp <= RESP_OKAY;
172     end
173     default: begin
174         s_axi_bresp <= RESP_SLVERR;
175     end
176 endcase
177 // Generate write response
178 s_axi_bvalid <= 1'b1;
179 // Clear captured transaction
180 aw_received <= 1'b0;
181 w_received <= 1'b0;
182 end
183 // Write Response Handshake
184
185 if (s_axi_bvalid && s_axi_bready) begin
186     s_axi_bvalid <= 1'b0;
187 end
188
189 // READ TRANSACTION
190
191 if (s_axi_arvalid && s_axi_arready) begin
192     case ((s_axi_araddr[31:4] == 28'd0 && s_axi_araddr[1:0] == 2'b00) ? s_axi_araddr[5:2]
+ : 4'hF)
193         4'd0: begin
194             s_axi_rdata <= reg0;
195             s_axi_rresp <= RESP_OKAY;
196         end
197         4'd1: begin
198             s_axi_rdata <= reg1;
199             s_axi_rresp <= RESP_OKAY;
200         end
201         4'd2: begin
202             s_axi_rdata <= reg2;
203             s_axi_rresp <= RESP_OKAY;
204         end
205         4'd3: begin
206             s_axi_rdata <= reg3;
207             s_axi_rresp <= RESP_OKAY;
208         end
209         default: begin
210             s_axi_rdata <= '0;
211             s_axi_rresp <= RESP_SLVERR;
212         end
213     endcase
214     // Generate read response
215     s_axi_rvalid <= 1'b1;
216 end
217
218 // Read Response Handshake
219
220 if (s_axi_rvalid && s_axi_rready) begin
221     s_axi_rvalid <= 1'b0;
222 end
223 end
224 endmodule

```

tb/master/axi4_lite_master.sv

197 lines

```
1 module axi4_lite_master #(
2     parameter ADDR_WIDTH = 32,
3     parameter DATA_WIDTH = 32
4 )
5     input logic          aclk,
6     input logic          aresetn,
7     output logic [ADDR_WIDTH-1:0] awaddr,
8     output logic          awvalid,
9     input logic          awready,
10    output logic [DATA_WIDTH-1:0] wdata,
11    output logic [DATA_WIDTH/8-1:0] wstrb,
12    output logic          wvalid,
13    input logic          wready,
14    input logic [1:0]      bresp,
15    input logic          bvalid,
16    output logic          bready,
17    output logic [ADDR_WIDTH-1:0] araddr,
18    output logic          arvalid,
19    input logic          arready,
20    input logic [DATA_WIDTH-1:0] rdata,
21    input logic [1:0]      rresp,
22    input logic          rvalid,
23    output logic          rready
24 );
25 // Last response codes seen, so tests can check them
26 logic [1:0] last_bresp = 2'b00;
27 logic [1:0] last_rresp = 2'b00;
28 integer b_delay = 0;
29 integer r_delay = 0;
30 integer fault_mode = 0;
31
32 initial begin
33     awaddr = '0;
34     awvalid = 1'b0;
35     wdata = '0;
36     wstrb = '0;
37     wvalid = 1'b0;
38     bready = 1'b0;
39     araddr = '0;
40     arvalid = 1'b0;
41     rready = 1'b0;
42 end
43
44 // TIMING RULE: this master changes its outputs and reads the slave's
45 // outputs only at the FALLING clock edge (middle of the cycle). The slave
46 // and the checkers work at the RISING edge. So nobody reads a signal at
47 // the moment it changes, and every simulator gives the same result.
48 // A transfer happens at the rising edge that follows a falling edge
49 // where VALID and READY were both high.
50
51 task automatic write(
52     input logic [ADDR_WIDTH-1:0] addr,
53     input logic [DATA_WIDTH-1:0] data,
54     input logic [DATA_WIDTH/8-1:0] strb
55 );
56     logic          seen;
57     logic [1:0] resp_seen;
58     begin
59         // Write address channel
60         @(negedge aclk);
61         awaddr = addr;
62         awvalid = 1'b1;
63         seen = awready;
64         @(posedge aclk);
65         while (!seen) begin
66             @(negedge aclk);
67             seen = awready;
68             if (!seen && fault_mode == 1) begin
69                 $display("[FAULT_INJECTED] mode=1 at time %0t", $time);
70                 fault_mode = 0; awvalid = 1'b0;
71                 @(posedge aclk); @(negedge aclk);
72                 awvalid = 1'b1; seen = awready;
```

```

73         end
74         if (!seen && fault_mode == 2) begin
75             $display("[FAULT_INJECTED] mode=2 at time %0t", $time);
76             fault_mode = 0; awaddr = awaddr ^ 32'h0000_0010;
77             @(posedge aclk); @(negedge aclk);
78             awaddr = addr; seen = awready;
79         end
80         @(posedge aclk);
81     end
82     @(negedge aclk);
83     awvalid = 1'b0;
84
85     // Write data channel
86     wdata = data;
87     wstrb = strb;
88     wvalid = 1'b1;
89     seen = wready;
90     @(posedge aclk);
91     while (!seen) begin
92         @(negedge aclk);
93         seen = wready;
94         if (!seen && fault_mode == 3) begin
95             $display("[FAULT_INJECTED] mode=3 at time %0t", $time);
96             fault_mode = 0; wvalid = 1'b0;
97             @(posedge aclk); @(negedge aclk);
98             wvalid = 1'b1; seen = wready;
99         end
100        if (!seen && fault_mode == 4) begin
101            $display("[FAULT_INJECTED] mode=4 at time %0t", $time);
102            fault_mode = 0; wdata = ~wdata;
103            @(posedge aclk); @(negedge aclk);
104            wdata = data; seen = wready;
105        end
106        if (!seen && fault_mode == 5) begin
107            $display("[FAULT_INJECTED] mode=5 at time %0t", $time);
108            fault_mode = 0; wstrb = ~wstrb;
109            @(posedge aclk); @(negedge aclk);
110            wstrb = strb; seen = wready;
111        end
112        @(posedge aclk);
113    end
114    @(negedge aclk);
115    wvalid = 1'b0;
116
117    // Write response channel
118    repeat (b_delay) @(negedge aclk);
119    bready = 1'b1;
120    seen = bvalid;
121    resp_seen = bresp;
122    @(posedge aclk);
123    while (!seen) begin
124        @(negedge aclk);
125        seen = bvalid;
126        resp_seen = bresp;
127        @(posedge aclk);
128    end
129    @(negedge aclk);
130    bready = 1'b0;
131    last_bresp = resp_seen;
132    if (resp_seen == 2'b00)
133        $display("[MASTER] WRITE SUCCESS ADDR=%h DATA=%h", addr, data);
134    else
135        $display("[MASTER] WRITE RESPONSE ERROR ADDR=%h BRESP=%b", addr, resp_seen);
136    end
137    endtask
138
139    task automatic read(
140        input logic [ADDR_WIDTH-1:0] addr,
141        output logic [DATA_WIDTH-1:0] data
142    );
143        logic                seen;
144        logic [1:0]          resp_seen;

```

```

145     logic [DATA_WIDTH-1:0] data_seen;
146     begin
147         // Read address channel
148         @(negedge aclk);
149         araddr = addr;
150         arvalid = 1'b1;
151         seen = arready;
152         @(posedge aclk);
153         while (!seen) begin
154             @(negedge aclk);
155             seen = arready;
156             if (!seen && fault_mode == 6) begin
157                 $display("[FAULT_INJECTED] mode=6 at time %0t", $time);
158                 fault_mode = 0; arvalid = 1'b0;
159                 @(posedge aclk); @(negedge aclk);
160                 arvalid = 1'b1; seen = arready;
161             end
162             if (!seen && fault_mode == 7) begin
163                 $display("[FAULT_INJECTED] mode=7 at time %0t", $time);
164                 fault_mode = 0; araddr = araddr ^ 32'h0000_0010;
165                 @(posedge aclk); @(negedge aclk);
166                 araddr = addr; seen = arready;
167             end
168             @(posedge aclk);
169         end
170         @(negedge aclk);
171         arvalid = 1'b0;
172
173         // Read data channel
174         repeat (r_delay) @(negedge aclk);
175         rready = 1'b1;
176         seen = rvalid;
177         data_seen = rdata;
178         resp_seen = rresp;
179         @(posedge aclk);
180         while (!seen) begin
181             @(negedge aclk);
182             seen = rvalid;
183             data_seen = rdata;
184             resp_seen = rresp;
185             @(posedge aclk);
186         end
187         @(negedge aclk);
188         rready = 1'b0;
189         data = data_seen;
190         last_rresp = resp_seen;
191         if (resp_seen == 2'b00)
192             $display("[MASTER] READ SUCCESS ADDR=%h DATA=%h", addr, data);
193         else
194             $display("[MASTER] READ RESPONSE ERROR ADDR=%h RRESP=%b", addr, resp_seen);
195     end
196 endtask
197 endmodule

```

tb/monitor/axi4_lite_monitor.sv

105 lines

```
1 module axi4_lite_monitor #(
2     parameter ADDR_WIDTH = 32,
3     parameter DATA_WIDTH = 32
4 )(
5     input logic          aclk,
6     input logic          aresetn,
7     // Write Address
8     input logic [ADDR_WIDTH-1:0] awaddr,
9     input logic          awvalid,
10    input logic          awready,
11    // Write Data
12    input logic [DATA_WIDTH-1:0] wdata,
13    input logic [DATA_WIDTH/8-1:0] wstrb,
14    input logic          wvalid,
15    input logic          wready,
16    // Write Response
17    input logic [1:0]      bresp,
18    input logic          bvalid,
19    input logic          bready,
20    // Read Address
21    input logic [ADDR_WIDTH-1:0] araddr,
22    input logic          arvalid,
23    input logic          arready,
24    // Read Data
25    input logic [DATA_WIDTH-1:0] rdata,
26    input logic [1:0]      rresp,
27    input logic          rvalid,
28    input logic          rready
29 );
30 // Stored Write Information
31
32 logic [ADDR_WIDTH-1:0] monitored_awaddr;
33 logic [DATA_WIDTH-1:0] monitored_wdata;
34 logic [DATA_WIDTH/8-1:0] monitored_wstrb;
35 logic aw_seen;
36 logic w_seen;
37 // Monitor
38
39 always @(posedge aclk) begin
40     if (!aresetn) begin
41         monitored_awaddr = '0;
42         monitored_wdata = '0;
43         monitored_wstrb = '0;
44         aw_seen = 1'b0;
45         w_seen = 1'b0;
46     end
47     else begin
48         // WRITE ADDRESS HANDSHAKE
49
50         if (awvalid && awready) begin
51             monitored_awaddr = awaddr;
52             aw_seen = 1'b1;
53             $display(
54                 "[MONITOR] AW HANDSHAKE: ADDR=%h",
55                 awaddr
56             );
57         end
58         // WRITE DATA HANDSHAKE
59
60         if (wvalid && wready) begin
61             monitored_wdata = wdata;
62             monitored_wstrb = wstrb;
63             w_seen = 1'b1;
64             $display(
65                 "[MONITOR] W HANDSHAKE: DATA=%h WSTRB=%b",
66                 wdata,
67                 wstrb
68             );
69         end
70
71         // WRITE RESPONSE
72     end
73 end
```

```

73         if (bvalid && bready) begin
74             $display(
75                 "[MONITOR] WRITE COMPLETE: ADDR=%h DATA=%h WSTRB=%b BRESP=%b",
76                 monitored_awaddr,
77                 monitored_wdata,
78                 monitored_wstrb,
79                 bresp
80             );
81             aw_seen = 1'b0;
82             w_seen  = 1'b0;
83         end
84
85         // READ ADDRESS HANDSHAKE
86
87         if (arvalid && arready) begin
88             $display(
89                 "[MONITOR] AR HANDSHAKE: ADDR=%h",
90                 araddr
91             );
92         end
93
94         // READ RESPONSE
95
96         if (rvalid && rready) begin
97             $display(
98                 "[MONITOR] READ COMPLETE: DATA=%h RRESP=%b",
99                 rdata,
100                rresp
101            );
102        end
103    end
104 end
105 endmodule

```

tb/scoreboard/axi4_lite_scoreboard.sv

170 lines

```
1 module axi4_lite_scoreboard #(
2     parameter ADDR_WIDTH = 32,
3     parameter DATA_WIDTH = 32
4 )(
5     input logic          aclk,
6     input logic          aresetn,
7     input logic [ADDR_WIDTH-1:0] awaddr,
8     input logic          awvalid,
9     input logic          awready,
10    input logic [DATA_WIDTH-1:0] wdata,
11    input logic [DATA_WIDTH/8-1:0] wstrb,
12    input logic          wvalid,
13    input logic          wready,
14    input logic [1:0]      bresp,
15    input logic          bvalid,
16    input logic          bready,
17    input logic [ADDR_WIDTH-1:0] araddr,
18    input logic          arvalid,
19    input logic          arready,
20    input logic [DATA_WIDTH-1:0] rdata,
21    input logic [1:0]      rresp,
22    input logic          rvalid,
23    input logic          rready
24 );
25     localparam [1:0] RESP_OKAY    = 2'b00;
26     localparam [1:0] RESP_SLVERR = 2'b10;
27
28     // Reference model: 4 registers
29     logic [DATA_WIDTH-1:0] ref_reg [0:3];
30
31     // Captured write (AW and W can arrive separately)
32     logic [ADDR_WIDTH-1:0] cap_awaddr;
33     logic [DATA_WIDTH-1:0] cap_wdata;
34     logic [DATA_WIDTH/8-1:0] cap_wstrb;
35     logic aw_got;
36     logic w_got;
37
38     // Expected results for the transaction in flight
39     logic [1:0]      exp_bresp;
40     logic          bresp_pending;
41     logic [ADDR_WIDTH-1:0] rd_addr;
42     logic [DATA_WIDTH-1:0] exp_rdata;
43     logic [1:0]      exp_rresp;
44     logic          rd_pending;
45
46     integer checks = 0;
47     integer errors = 0;
48     integer wi, ri, bb;
49     logic [DATA_WIDTH-1:0] tmp;
50
51     // Address map from the spec: four word registers at 0x0, 0x4, 0x8, 0xC.
52     // Returns -1 for any other address.
53     function automatic integer idx_of(input logic [ADDR_WIDTH-1:0] a);
54         case (a)
55             32'h0000_0000: idx_of = 0;
56             32'h0000_0004: idx_of = 1;
57             32'h0000_0008: idx_of = 2;
58             32'h0000_000C: idx_of = 3;
59             default:      idx_of = -1;
60         endcase
61     endfunction
62
63     always @(posedge aclk) begin
64         if (!aresetn) begin
65             for (bb = 0; bb < 4; bb = bb + 1)
66                 ref_reg[bb] = '0;
67             aw_got = 1'b0;
68             w_got = 1'b0;
69             bresp_pending = 1'b0;
70             rd_pending = 1'b0;
71         end
72         else begin
```

```

73      // Capture write address and write data handshakes
74      if (awvalid && awready) begin
75          cap_awaddr = awaddr;
76          aw_got = 1'b1;
77      end
78      if (wvalid && wready) begin
79          cap_wdata = wdata;
80          cap_wstrb = wstrb;
81          w_got = 1'b1;
82      end
83
84      // When both have arrived, update the model and the expected response
85      if (aw_got && w_got) begin
86          wi = idx_of(cap_awaddr);
87          if (wi >= 0) begin
88              tmp = ref_reg[wi];
89              for (bb = 0; bb < 4; bb = bb + 1)
90                  if (cap_wstrb[bb])
91                      tmp[8*bb+: 8] = cap_wdata[8*bb+: 8];
92              ref_reg[wi] = tmp;
93              exp_bresp = RESP_OKAY;
94          end
95          else begin
96              exp_bresp = RESP_SLVERR;
97          end
98          bresp_pending = 1'b1;
99          aw_got = 1'b0;
100         w_got = 1'b0;
101     end
102
103     // Check the write response
104     if (bvalid && bready) begin
105         checks = checks + 1;
106         if (!bresp_pending) begin
107             errors = errors + 1;
108             $display("[SCOREBOARD] ERROR: B handshake with no write pending");
109         end
110         else begin
111             if (bresp === exp_bresp)
112                 $display("[SCOREBOARD] WRITE RESPONSE PASS: BRESP=%b", bresp);
113             else begin
114                 errors = errors + 1;
115                 $display("[SCOREBOARD] WRITE RESPONSE FAIL: EXPECTED=%b GOT=%b",
116                     exp_bresp, bresp);
117             end
118             bresp_pending = 1'b0;
119         end
120     end
121
122     // Capture the read request and work out what the answer should be
123     if (arvalid && arready) begin
124         rd_addr = araddr;
125         ri = idx_of(araddr);
126         if (ri >= 0) begin
127             exp_rdata = ref_reg[ri];
128             exp_rresp = RESP_OKAY;
129         end
130         else begin
131             exp_rdata = '0;
132             exp_rresp = RESP_SLVERR;
133         end
134         rd_pending = 1'b1;
135     end
136
137     // Check the read response
138     if (rvalid && rready) begin
139         checks = checks + 1;
140         if (!rd_pending) begin
141             errors = errors + 1;
142             $display("[SCOREBOARD] ERROR: R handshake with no read pending");
143         end
144         else begin

```

```

145         if (rdata === exp_rdata && rresp === exp_rresp)
146             $display("[SCOREBOARD] READ PASS: ADDR=%h DATA=%h RRESP=%b",
147                 rd_addr, rdata, rresp);
148         else begin
149             errors = errors + 1;
150             $display("[SCOREBOARD] READ FAIL: ADDR=%h EXP_DATA=%h EXP_RRESP=%b
+ GOT_DATA=%h GOT_RRESP=%b",
151                 rd_addr, exp_rdata, exp_rresp, rdata, rresp);
152         end
153         rd_pending = 1'b0;
154     end
155 end
156 end
157 end
158
159 final begin
160     $display("");
161     $display("=====");
162     $display("          SCOREBOARD SUMMARY");
163     $display("=====");
164     $display("CHECKS = %0d  ERRORS = %0d", checks, errors);
165     if (errors == 0 && checks > 0 && !bresp_pending && !rd_pending && !aw_got && !w_got)
166         $display("SCOREBOARD RESULT: PASS");
167     else
168         $display("SCOREBOARD RESULT: FAIL");
169     end
170 endmodule

```

tb/coverage/axi4_lite_coverage.sv

245 lines

```
1 module axi4_lite_coverage #(
2     parameter ADDR_WIDTH = 32,
3     parameter DATA_WIDTH = 32
4 ) (
5     input logic                aclk,
6     input logic                aresetn,
7     input logic [ADDR_WIDTH-1:0] awaddr,
8     input logic                awvalid,
9     input logic                awready,
10    input logic [DATA_WIDTH-1:0] wdata,
11    input logic [DATA_WIDTH/8-1:0] wstrb,
12    input logic                wvalid,
13    input logic                wready,
14    input logic [1:0]          bresp,
15    input logic                bvalid,
16    input logic                bready,
17    input logic [ADDR_WIDTH-1:0] araddr,
18    input logic                arvalid,
19    input logic                arready,
20    input logic [DATA_WIDTH-1:0] rdata,
21    input logic [1:0]          rresp,
22    input logic                rvalid,
23    input logic                rready
24 );
25 integer write_reg0_count;
26 integer write_reg1_count;
27 integer write_reg2_count;
28 integer write_reg3_count;
29 integer write_invalid_count;
30 integer read_reg0_count;
31 integer read_reg1_count;
32 integer read_reg2_count;
33 integer read_reg3_count;
34 integer read_invalid_count;
35 integer wstrb_1111_count;
36 integer wstrb_0001_count;
37 integer wstrb_0010_count;
38 integer wstrb_0100_count;
39 integer wstrb_1000_count;
40 integer wstrb_partial_count;
41 integer wstrb_0000_count;
42 integer bresp_okay_count;
43 integer bresp_slvterr_count;
44 integer rresp_okay_count;
45 integer rresp_slvterr_count;
46 integer total_items;
47 integer covered_items;
48 real coverage_percentage;
49 // INITIALIZATION
50
51 initial begin
52     write_reg0_count = 0;
53     write_reg1_count = 0;
54     write_reg2_count = 0;
55     write_reg3_count = 0;
56     write_invalid_count = 0;
57     read_reg0_count = 0;
58     read_reg1_count = 0;
59     read_reg2_count = 0;
60     read_reg3_count = 0;
61     read_invalid_count = 0;
62     wstrb_1111_count = 0;
63     wstrb_0001_count = 0;
64     wstrb_0010_count = 0;
65     wstrb_0100_count = 0;
66     wstrb_1000_count = 0;
67     wstrb_partial_count = 0;
68     wstrb_0000_count = 0;
69     bresp_okay_count = 0;
70     bresp_slvterr_count = 0;
71     rresp_okay_count = 0;
72     rresp_slvterr_count = 0;
```

```

73     end
74     // COVERAGE SAMPLING
75
76     always @(posedge aclk) begin
77         if (aresetn) begin
78             // WRITE ADDRESS
79             if (awvalid && awready) begin
80                 case ((awaddr[31:4] == 28'd0 && awaddr[1:0] == 2'b00) ? awaddr[5:2] : 4'hF)
81                     4'd0:
82                         write_reg0_count = write_reg0_count + 1;
83                     4'd1:
84                         write_reg1_count = write_reg1_count + 1;
85                     4'd2:
86                         write_reg2_count = write_reg2_count + 1;
87                     4'd3:
88                         write_reg3_count = write_reg3_count + 1;
89                     default:
90                         write_invalid_count =
91                             write_invalid_count + 1;
92                 endcase
93             end
94             // WRITE STROBE
95             if (wvalid && wready) begin
96                 case (wstrb)
97                     4'b1111:
98                         wstrb_1111_count =
99                             wstrb_1111_count + 1;
100                    4'b0001:
101                        wstrb_0001_count =
102                            wstrb_0001_count + 1;
103                    4'b0010:
104                        wstrb_0010_count =
105                            wstrb_0010_count + 1;
106                    4'b0100:
107                        wstrb_0100_count =
108                            wstrb_0100_count + 1;
109                    4'b1000:
110                        wstrb_1000_count =
111                            wstrb_1000_count + 1;
112                    4'b0000:
113                        wstrb_0000_count =
114                            wstrb_0000_count + 1;
115                    default:
116                        wstrb_partial_count =
117                            wstrb_partial_count + 1;
118                endcase
119            end
120            // WRITE RESPONSE
121            if (bvalid && bready) begin
122                if (bresp == 2'b00)
123                    bresp_okay_count =
124                        bresp_okay_count + 1;
125                else if (bresp == 2'b10)
126                    bresp_slvrr_count =
127                        bresp_slvrr_count + 1;
128            end
129            // READ ADDRESS
130            if (arvalid && arready) begin
131                case ((araddr[31:4] == 28'd0 && araddr[1:0] == 2'b00) ? araddr[5:2] : 4'hF)
132                    4'd0:
133                        read_reg0_count =
134                            read_reg0_count + 1;
135                    4'd1:
136                        read_reg1_count =
137                            read_reg1_count + 1;
138                    4'd2:
139                        read_reg2_count =
140                            read_reg2_count + 1;
141                    4'd3:
142                        read_reg3_count =
143                            read_reg3_count + 1;
144                    default:

```

```

145             read_invalid_count =
146                 read_invalid_count + 1;
147         endcase
148     end
149     // READ RESPONSE
150     if (rvalid && rready) begin
151         if (rresp == 2'b00)
152             rresp_okay_count =
153                 rresp_okay_count + 1;
154         else if (rresp == 2'b10)
155             rresp_slvrr_count =
156                 rresp_slvrr_count + 1;
157     end
158 end
159 end
160 // FINAL COVERAGE REPORT
161
162 final begin
163     total_items = 21;
164     covered_items = 0;
165     if (write_reg0_count > 0)
166         covered_items = covered_items + 1;
167     if (write_reg1_count > 0)
168         covered_items = covered_items + 1;
169     if (write_reg2_count > 0)
170         covered_items = covered_items + 1;
171     if (write_reg3_count > 0)
172         covered_items = covered_items + 1;
173     if (write_invalid_count > 0)
174         covered_items = covered_items + 1;
175     if (read_reg0_count > 0)
176         covered_items = covered_items + 1;
177     if (read_reg1_count > 0)
178         covered_items = covered_items + 1;
179     if (read_reg2_count > 0)
180         covered_items = covered_items + 1;
181     if (read_reg3_count > 0)
182         covered_items = covered_items + 1;
183     if (read_invalid_count > 0)
184         covered_items = covered_items + 1;
185     if (wstrb_1111_count > 0)
186         covered_items = covered_items + 1;
187     if (wstrb_0001_count > 0)
188         covered_items = covered_items + 1;
189     if (wstrb_0010_count > 0)
190         covered_items = covered_items + 1;
191     if (wstrb_0100_count > 0)
192         covered_items = covered_items + 1;
193     if (wstrb_1000_count > 0)
194         covered_items = covered_items + 1;
195     if (wstrb_partial_count > 0)
196         covered_items = covered_items + 1;
197     if (wstrb_0000_count > 0)
198         covered_items = covered_items + 1;
199     if (bresp_okay_count > 0)
200         covered_items = covered_items + 1;
201     if (bresp_slvrr_count > 0)
202         covered_items = covered_items + 1;
203     if (rresp_okay_count > 0)
204         covered_items = covered_items + 1;
205     if (rresp_slvrr_count > 0)
206         covered_items = covered_items + 1;
207     coverage_percentage =
208         (covered_items * 100.0) / total_items;
209     $display("");
210     $display("=====");
211     $display("          AXI4-LITE FUNCTIONAL COVERAGE");
212     $display("=====");
213     $display("WRITE REG0      = %0d", write_reg0_count);
214     $display("WRITE REG1      = %0d", write_reg1_count);
215     $display("WRITE REG2      = %0d", write_reg2_count);
216     $display("WRITE REG3      = %0d", write_reg3_count);

```

```

217     $display("WRITE INVALID    = %0d", write_invalid_count);
218     $display("");
219     $display("READ REG0       = %0d", read_reg0_count);
220     $display("READ REG1       = %0d", read_reg1_count);
221     $display("READ REG2       = %0d", read_reg2_count);
222     $display("READ REG3       = %0d", read_reg3_count);
223     $display("READ INVALID    = %0d", read_invalid_count);
224     $display("");
225     $display("WSTRB 1111      = %0d", wstrb_1111_count);
226     $display("WSTRB 0001      = %0d", wstrb_0001_count);
227     $display("WSTRB 0010      = %0d", wstrb_0010_count);
228     $display("WSTRB 0100      = %0d", wstrb_0100_count);
229     $display("WSTRB 1000      = %0d", wstrb_1000_count);
230     $display("WSTRB PARTIAL   = %0d", wstrb_partial_count);
231     $display("WSTRB 0000      = %0d", wstrb_0000_count);
232     $display("");
233     $display("BRESP OKAY       = %0d", bresp_okay_count);
234     $display("BRESP SLVERR    = %0d", bresp_slvrr_count);
235     $display("RRESP OKAY       = %0d", rresp_okay_count);
236     $display("RRESP SLVERR    = %0d", rresp_slvrr_count);
237     $display("");
238     $display("COVERED BINS    = %0d / %0d",
239           covered_items,
240           total_items);
241     $display("COVERAGE      = %0.2f%%",
242           coverage_percentage);
243     $display("=====");
244     end
245 endmodule

```

tb/assertions/axi4_lite_sva.sv

154 lines

```
1 `define SVA_FAIL(NAME) begin sva_fail_total = sva_fail_total + 1; $display("[SVA_FAIL] %s at time
+ %0t", NAME, $time); end
2
3 module axi4_lite_sva #(
4     parameter ADDR_WIDTH = 32,
5     parameter DATA_WIDTH = 32,
6     parameter B_MAX_WAIT = 16,
7     parameter R_MAX_WAIT = 16
8 ) (
9     input logic          aclk,
10    input logic          aresetn,
11    input logic [ADDR_WIDTH-1:0] awaddr,
12    input logic          awvalid,
13    input logic          awready,
14    input logic [DATA_WIDTH-1:0] wdata,
15    input logic [DATA_WIDTH/8-1:0] wstrb,
16    input logic          wvalid,
17    input logic          wready,
18    input logic [1:0]      bresp,
19    input logic          bvalid,
20    input logic          bready,
21    input logic [ADDR_WIDTH-1:0] araddr,
22    input logic          arvalid,
23    input logic          arready,
24    input logic [DATA_WIDTH-1:0] rdata,
25    input logic [1:0]      rresp,
26    input logic          rvalid,
27    input logic          rready
28 );
29     integer sva_fail_total = 0;
30
31     // ----- helper counters (handshakes seen) -----
32     integer aw_cnt = 0, w_cnt = 0, b_cnt = 0, ar_cnt = 0, r_cnt = 0;
33     integer b_wait = 0, r_wait = 0;
34
35     wire b_pending = (aw_cnt > b_cnt) && (w_cnt > b_cnt);
36     wire r_pending = (ar_cnt > r_cnt);
37
38     always @(posedge aclk) begin
39         if (!aresetn) begin
40             aw_cnt <= 0; w_cnt <= 0; b_cnt <= 0;
41             ar_cnt <= 0; r_cnt <= 0;
42             b_wait <= 0; r_wait <= 0;
43         end
44         else begin
45             if (awvalid && awready) aw_cnt <= aw_cnt + 1;
46             if (wvalid && wready) w_cnt <= w_cnt + 1;
47             if (bvalid && bready) b_cnt <= b_cnt + 1;
48             if (arvalid && arready) ar_cnt <= ar_cnt + 1;
49             if (rvalid && rready) r_cnt <= r_cnt + 1;
50             if (b_pending && !bvalid) b_wait <= b_wait + 1; else b_wait <= 0;
51             if (r_pending && !rvalid) r_wait <= r_wait + 1; else r_wait <= 0;
52         end
53     end
54
55     // ----- stability rules: sender must hold VALID and payload -----
56     A01_AWVALID_HOLD: assert property (@(posedge aclk)
57         (aresetn && awvalid && !awready) |>= (!aresetn || awvalid))
58         else `SVA_FAIL("A01_AWVALID_HOLD")
59     A02_AWADDR_STABLE: assert property (@(posedge aclk)
60         (aresetn && awvalid && !awready) |>= (!aresetn || $stable(awaddr)))
61         else `SVA_FAIL("A02_AWADDR_STABLE")
62     A03_WVALID_HOLD: assert property (@(posedge aclk)
63         (aresetn && wvalid && !wready) |>= (!aresetn || wvalid))
64         else `SVA_FAIL("A03_WVALID_HOLD")
65     A04_WDATA_STABLE: assert property (@(posedge aclk)
66         (aresetn && wvalid && !wready) |>= (!aresetn || $stable(wdata)))
67         else `SVA_FAIL("A04_WDATA_STABLE")
68     A05_WSTRB_STABLE: assert property (@(posedge aclk)
69         (aresetn && wvalid && !wready) |>= (!aresetn || $stable(wstrb)))
70         else `SVA_FAIL("A05_WSTRB_STABLE")
71     A06_ARVALID_HOLD: assert property (@(posedge aclk)
72         (aresetn && arvalid && !arready) |>= (!aresetn || arvalid))
```

```

73     else `SVA_FAIL("A06_ARVALID_HOLD")
74 A07_ARADDR_STABLE: assert property (@(posedge aclk)
75     (aresetn && arvalid && !arready) |>= (!aresetn || $stable(araddr)))
76     else `SVA_FAIL("A07_ARADDR_STABLE")
77 A08_BVALID_HOLD: assert property (@(posedge aclk)
78     (aresetn && bvalid && !bready) |>= (!aresetn || bvalid))
79     else `SVA_FAIL("A08_BVALID_HOLD")
80 A09_BRESP_STABLE: assert property (@(posedge aclk)
81     (aresetn && bvalid && !bready) |>= (!aresetn || $stable(bresp)))
82     else `SVA_FAIL("A09_BRESP_STABLE")
83 A10_RVALID_HOLD: assert property (@(posedge aclk)
84     (aresetn && rvalid && !rready) |>= (!aresetn || rvalid))
85     else `SVA_FAIL("A10_RVALID_HOLD")
86 A11_RDATA_STABLE: assert property (@(posedge aclk)
87     (aresetn && rvalid && !rready) |>= (!aresetn || $stable(rdata)))
88     else `SVA_FAIL("A11_RDATA_STABLE")
89 A12_RRESP_STABLE: assert property (@(posedge aclk)
90     (aresetn && rvalid && !rready) |>= (!aresetn || $stable(rresp)))
91     else `SVA_FAIL("A12_RRESP_STABLE")
92
93 // ----- response legality (this design: OKAY or SLVERR only) -----
94 A13_BRESP_LEGAL: assert property (@(posedge aclk)
95     (aresetn && bvalid) |>= (bresp == 2'b00 || bresp == 2'b10))
96     else `SVA_FAIL("A13_BRESP_LEGAL")
97 A14_RRESP_LEGAL: assert property (@(posedge aclk)
98     (aresetn && rvalid) |>= (rresp == 2'b00 || rresp == 2'b10))
99     else `SVA_FAIL("A14_RRESP_LEGAL")
100
101 // ----- reset -----
102 A15_RESET_RESP_LOW: assert property (@(posedge aclk)
103     (!aresetn) |>= (!bvalid && !rvalid))
104     else `SVA_FAIL("A15_RESET_RESP_LOW")
105
106 // ----- no response without a request -----
107 A16_NO_B_WITHOUT_WRITE: assert property (@(posedge aclk)
108     (aresetn && bvalid) |>= (aw_cnt > b_cnt && w_cnt > b_cnt))
109     else `SVA_FAIL("A16_NO_B_WITHOUT_WRITE")
110 A17_NO_R_WITHOUT_READ: assert property (@(posedge aclk)
111     (aresetn && rvalid) |>= (ar_cnt > r_cnt))
112     else `SVA_FAIL("A17_NO_R_WITHOUT_READ")
113
114 // ----- bounded response (counter based; reports once) -----
115 always @(posedge aclk) begin
116     if (aresetn) begin
117         A18_B_TIMEOUT: assert (b_wait != B_MAX_WAIT + 1)
118         else `SVA_FAIL("A18_B_TIMEOUT")
119         A19_R_TIMEOUT: assert (r_wait != R_MAX_WAIT + 1)
120         else `SVA_FAIL("A19_R_TIMEOUT")
121     end
122 end
123
124 // ----- sanity counters: did each rule's trigger ever happen? -----
125 integer n_aw_stall = 0, n_w_stall = 0, n_ar_stall = 0;
126 integer n_b_stall = 0, n_r_stall = 0, n_b_hs = 0, n_r_hs = 0;
127 always @(posedge aclk) begin
128     if (aresetn) begin
129         if (awvalid && !awready) n_aw_stall = n_aw_stall + 1;
130         if (wvalid && !wready) n_w_stall = n_w_stall + 1;
131         if (arvalid && !arready) n_ar_stall = n_ar_stall + 1;
132         if (bvalid && !bready) n_b_stall = n_b_stall + 1;
133         if (rvalid && !rready) n_r_stall = n_r_stall + 1;
134         if (bvalid && bready) n_b_hs = n_b_hs + 1;
135         if (rvalid && rready) n_r_hs = n_r_hs + 1;
136     end
137 end
138
139 final begin
140     $display("");
141     $display("=====");
142     $display("                SVA SUMMARY");
143     $display("=====");
144     $display("SVA FAILS = %0d", sva_fail_total);
145
146     $display("SANITY: AW stall cycles = %0d", n_aw_stall);
147     $display("SANITY: W  stall cycles = %0d", n_w_stall);
148     $display("SANITY: AR stall cycles = %0d", n_ar_stall);
149     $display("SANITY: B  stall cycles = %0d", n_b_stall);
150     $display("SANITY: R  stall cycles = %0d", n_r_stall);
151     $display("SANITY: B handshakes  = %0d", n_b_hs);
152     $display("SANITY: R handshakes  = %0d", n_r_hs);
153     $display("(stall count 0 means the matching stability rules were never exercised)");
154 end
endmodule

```

tb/tb_top.sv

456 lines

```
1  `timescale 1ns/1ps
2  module tb_top;
3      parameter ADDR_WIDTH = 32;
4      parameter DATA_WIDTH = 32;
5      //=====
6      // CLOCK AND RESET
7      //=====
8      logic aclk;
9      logic aresetn;
10     //=====
11     // AXI4-LITE SIGNALS
12     //=====
13     // Write Address
14     wire [ADDR_WIDTH-1:0] awaddr;
15     wire awvalid;
16     wire awready;
17     // Write Data
18     wire [DATA_WIDTH-1:0] wdata;
19     wire [DATA_WIDTH/8-1:0] wstrb;
20     wire wvalid;
21     wire wready;
22     // Write Response
23     wire [1:0] bresp;
24     wire bvalid;
25     wire bready;
26     // Read Address
27     wire [ADDR_WIDTH-1:0] araddr;
28     wire arvalid;
29     wire arready;
30     // Read Data
31     wire [DATA_WIDTH-1:0] rdata;
32     wire [1:0] rresp;
33     wire rvalid;
34     wire rready;
35     // CLOCK GENERATION
36     // 100 MHz clock
37
38     initial begin
39         aclk = 1'b0;
40         forever #5 aclk = ~aclk;
41     end
42     // RESET GENERATION
43
44     initial begin
45         aresetn = 1'b0;
46         repeat (5)
47             @(posedge aclk);
48         aresetn = 1'b1;
49         $display("");
50         $display("=====");
51         $display("RESET RELEASED");
52         $display("=====");
53         $display("");
54     end
55     // DUT
56
57     axi4_lite_slave #(
58         .ADDR_WIDTH(ADDR_WIDTH),
59         .DATA_WIDTH(DATA_WIDTH)
60     ) dut (
61         .aclk (aclk),
62         .aresetn (aresetn),
63         .s_axi_awaddr (awaddr),
64         .s_axi_awvalid (awvalid),
65         .s_axi_awready (awready),
66         .s_axi_wdata (wdata),
67         .s_axi_wstrb (wstrb),
68         .s_axi_wvalid (wvalid),
69         .s_axi_wready (wready),
70         .s_axi_bresp (bresp),
71         .s_axi_bvalid (bvalid),
72         .s_axi_bready (bready),
```

```

73     .s_axi_araddr  (araddr),
74     .s_axi_arvalid (arvalid),
75     .s_axi_arready (arready),
76     .s_axi_rdata   (rdata),
77     .s_axi_rresp    (rresp),
78     .s_axi_rvalid   (rvalid),
79     .s_axi_rready   (rready)
80 );
81
82 // MASTER / BFM
83
84 axi4_lite_master #(
85     .ADDR_WIDTH(ADDR_WIDTH),
86     .DATA_WIDTH(DATA_WIDTH)
87 ) master (
88     .aclk      (aclk),
89     .aresetn   (aresetn),
90     .awaddr    (awaddr),
91     .awvalid   (awvalid),
92     .awready   (awready),
93     .wdata     (wdata),
94     .wstrb     (wstrb),
95     .wvalid    (wvalid),
96     .wready    (wready),
97     .bresp     (bresp),
98     .bvalid    (bvalid),
99     .bready    (bready),
100    .araddr    (araddr),
101    .arvalid   (arvalid),
102    .arready   (arready),
103    .rdata     (rdata),
104    .rresp     (rresp),
105    .rvalid    (rvalid),
106    .rready    (rready)
107 );
108
109 // MONITOR
110
111 axi4_lite_monitor #(
112     .ADDR_WIDTH(ADDR_WIDTH),
113     .DATA_WIDTH(DATA_WIDTH)
114 ) monitor (
115     .aclk      (aclk),
116     .aresetn   (aresetn),
117     .awaddr    (awaddr),
118     .awvalid   (awvalid),
119     .awready   (awready),
120     .wdata     (wdata),
121     .wstrb     (wstrb),
122     .wvalid    (wvalid),
123     .wready    (wready),
124     .bresp     (bresp),
125     .bvalid    (bvalid),
126     .bready    (bready),
127     .araddr    (araddr),
128     .arvalid   (arvalid),
129     .arready   (arready),
130     .rdata     (rdata),
131     .rresp     (rresp),
132     .rvalid    (rvalid),
133     .rready    (rready)
134 );
135 // SCOREBOARD
136
137 axi4_lite_scoreboard #(
138     .ADDR_WIDTH(ADDR_WIDTH),
139     .DATA_WIDTH(DATA_WIDTH)
140 ) scoreboard (
141     .aclk      (aclk),
142     .aresetn   (aresetn),
143     .awaddr    (awaddr),
144     .awvalid   (awvalid),

```

```

145     .awready    (awready),
146     .wdata      (wdata),
147     .wstrb      (wstrb),
148     .wvalid     (wvalid),
149     .wready     (wready),
150     .bresp      (bresp),
151     .bvalid     (bvalid),
152     .bready     (bready),
153     .araddr     (araddr),
154     .arvalid    (arvalid),
155     .arready    (arready),
156     .rdata      (rdata),
157     .rresp      (rresp),
158     .rvalid     (rvalid),
159     .rready     (rready)
160 );
161 // COVERAGE
162
163 axi4_lite_coverage #(
164     .ADDR_WIDTH(ADDR_WIDTH),
165     .DATA_WIDTH(DATA_WIDTH)
166 ) coverage (
167     .aclk        (aclk),
168     .aresetn     (aresetn),
169     .awaddr      (awaddr),
170     .awvalid     (awvalid),
171     .awready     (awready),
172     .wdata       (wdata),
173     .wstrb       (wstrb),
174     .wvalid      (wvalid),
175     .wready      (wready),
176     .bresp       (bresp),
177     .bvalid      (bvalid),
178     .bready      (bready),
179     .araddr      (araddr),
180     .arvalid     (arvalid),
181     .arready     (arready),
182     .rdata       (rdata),
183     .rresp       (rresp),
184     .rvalid      (rvalid),
185     .rready      (rready)
186 );
187
188 // FAULT INJECTION MODULE
189
190 // TEST VARIABLE
191
192 // ---- extra test bookkeeping ----
193 integer ext_pass = 0;
194 integer ext_fail = 0;
195 integer fault_sel = 0;
196 integer rseed = 1;
197 integer rnd_cnt = 0;
198 integer rnd_n = 0;
199 integer rnd_sel = 0;
200 logic [31:0] rnd_addr;
201 logic [31:0] rnd_data;
202 logic [3:0]  rnd_strb;
203 // Simple xorshift32 random generator (same result in every simulator)
204 reg [31:0] rng;
205 function automatic [31:0] xorshift(input [31:0] s);
206     reg [31:0] x;
207     begin
208         x = s;
209         x = x ^ (x << 13);
210         x = x ^ (x >> 17);
211         x = x ^ (x << 5);
212         xorshift = x;
213     end
214 endfunction
215 `define CHK(EXP, ACT, NAME) \
216     if ((ACT) === (EXP)) begin \

```

```

217     $display("[TEST] %s PASS", NAME); \
218     ext_pass = ext_pass + 1; \
219 end else begin \
220     $display("[TEST] %s FAIL EXPECTED=%h ACTUAL=%h", NAME, (EXP), (ACT)); \
221     ext_fail = ext_fail + 1; \
222 end
223
224 logic [DATA_WIDTH-1:0] read_data;
225 // BASIC TEST
226 initial begin
227     wait(aresetn == 1'b1);
228     @(posedge aclk);
229     $display("");
230     $display("=====");
231     $display("          AXI4-LITE BASIC TEST START");
232     $display("=====");
233     $display("");
234
235     // TEST 1 : REG0
236
237     master.write(
238         32'h0000_0000,
239         32'h1234_5678,
240         4'b1111
241     );
242     master.read(
243         32'h0000_0000,
244         read_data
245     );
246     if (read_data === 32'h1234_5678)
247         $display("[TEST] REG0 PASS");
248     else
249         $error(
250             "[TEST] REG0 FAIL EXPECTED=%h ACTUAL=%h",
251             32'h1234_5678,
252             read_data
253         );
254     // TEST 2 : REG1
255
256     master.write(
257         32'h0000_0004,
258         32'hAABB_CCDD,
259         4'b1111
260     );
261     master.read(
262         32'h0000_0004,
263         read_data
264     );
265     if (read_data === 32'hAABB_CCDD)
266         $display("[TEST] REG1 PASS");
267     else
268         $error(
269             "[TEST] REG1 FAIL EXPECTED=%h ACTUAL=%h",
270             32'hAABB_CCDD,
271             read_data
272         );
273     // TEST 3 : REG2
274
275     master.write(
276         32'h0000_0008,
277         32'hDEAD_BEEF,
278         4'b1111
279     );
280     master.read(
281         32'h0000_0008,
282         read_data
283     );
284     if (read_data === 32'hDEAD_BEEF)
285         $display("[TEST] REG2 PASS");
286     else
287         $error(
288             "[TEST] REG2 FAIL EXPECTED=%h ACTUAL=%h",

```

```

289         32'hDEAD_BEEF,
290         read_data
291     );
292
293     // TEST 4 : REG3
294
295     master.write(
296         32'h0000_000C,
297         32'hCAFE_BABE,
298         4'b1111
299     );
300     master.read(
301         32'h0000_000C,
302         read_data
303     );
304     if (read_data === 32'hCAFE_BABE)
305         $display("[TEST] REG3 PASS");
306     else
307         $error(
308             "[TEST] REG3 FAIL EXPECTED=%h ACTUAL=%h",
309             32'hCAFE_BABE,
310             read_data
311         );
312     // =====
313     // EXTRA TESTS: coverage closure plus checks
314     // =====
315     master.write(32'h0000_0000, 32'hFFFF_FFFF, 4'b1111);
316     master.write(32'h0000_0000, 32'h1122_3344, 4'b0001);
317     `CHK(2'b00, master.last_bresp, "WRITE BRESP=OKAY")
318     master.read (32'h0000_0000, read_data);
319     `CHK(32'hFFFF_FF44, read_data, "WSTRB 0001")
320
321     master.write(32'h0000_0000, 32'h1122_3344, 4'b0010);
322     master.read (32'h0000_0000, read_data);
323     `CHK(32'hFFFF_3344, read_data, "WSTRB 0010")
324
325     master.write(32'h0000_0000, 32'h1122_3344, 4'b0100);
326     master.read (32'h0000_0000, read_data);
327     `CHK(32'hFF22_3344, read_data, "WSTRB 0100")
328
329     master.write(32'h0000_0000, 32'h1122_3344, 4'b1000);
330     master.read (32'h0000_0000, read_data);
331     `CHK(32'h1122_3344, read_data, "WSTRB 1000")
332
333     master.write(32'h0000_0000, 32'hAABB_CCDD, 4'b0101);
334     master.read (32'h0000_0000, read_data);
335     `CHK(32'h11BB_33DD, read_data, "WSTRB 0101 partial")
336
337     master.write(32'h0000_0000, 32'h0000_0000, 4'b0000);
338     master.read (32'h0000_0000, read_data);
339     `CHK(32'h11BB_33DD, read_data, "WSTRB 0000 no change")
340
341     master.write(32'h1000_0000, 32'hDEAD_BEEF, 4'b1111);
342     `CHK(2'b10, master.last_bresp, "INVALID WRITE 10000000 BRESP=SLVERR")
343     master.read (32'h0000_0000, read_data);
344     `CHK(32'h11BB_33DD, read_data, "INVALID WRITE 10000000")
345
346     master.write(32'h0000_0040, 32'hDEAD_BEEF, 4'b1111);
347     `CHK(2'b10, master.last_bresp, "INVALID WRITE 00000040 BRESP=SLVERR")
348     master.read (32'h0000_0000, read_data);
349     `CHK(32'h11BB_33DD, read_data, "INVALID WRITE 00000040")
350
351     master.read (32'h1000_0000, read_data);
352     `CHK(2'b10, master.last_rresp, "INVALID READ RRESP=SLVERR")
353     `CHK(32'h0000_0000, read_data, "INVALID READ data zero")
354
355     // --- backpressure: master delays BREADY and RREADY (creates stalls)
356     master.b_delay = 3;
357     master.r_delay = 3;
358     master.write(32'h0000_0004, 32'hA5A5_5A5A, 4'b1111);
359     `CHK(2'b00, master.last_bresp, "BACKPRESSURE WRITE BRESP=OKAY")
360     master.read (32'h0000_0004, read_data);

```

```

361 `CHK(32'hA5A5_5A5A, read_data, "BACKPRESSURE READ DATA")
362 master.b_delay = 0;
363 master.r_delay = 0;
364
365 // --- stall test: slave holds READY low for 4 cycles (exercises A01-A07)
366 if (!$value$plusargs("FAULT=%d", fault_sel)) fault_sel = 0;
367 dut.ready_delay = 4;
368 master.write(32'h0000_0008, 32'h5555_AAAA, 4'b1111);
369 `CHK(2'b00, master.last_bresp, "STALL WRITE BRESP=OKAY")
370 master.read (32'h0000_0008, read_data);
371 `CHK(32'h5555_AAAA, read_data, "STALL READ DATA")
372 // --- optional injected fault (only when run with +FAULT=n)
373 if (fault_sel != 0) begin
374     master.fault_mode = fault_sel;
375     if (fault_sel <= 5) master.write(32'h0000_000C, 32'h1234_ABCD, 4'b1111);
376     else master.read (32'h0000_000C, read_data);
377     master.fault_mode = 0;
378 end
379 dut.ready_delay = 0;
380
381 // --- unaligned addresses are invalid (SLVERR, no effect)
382 master.write(32'h0000_0001, 32'hBAD0_BAD0, 4'b1111);
383 `CHK(2'b10, master.last_bresp, "UNALIGNED WRITE 0x1 BRESP=SLVERR")
384 master.read (32'h0000_0000, read_data);
385 `CHK(32'h11BB_33DD, read_data, "UNALIGNED WRITE no effect")
386 master.read (32'h0000_0002, read_data);
387 `CHK(2'b10, master.last_rresp, "UNALIGNED READ 0x2 RRESP=SLVERR")
388 `CHK(32'h0000_0000, read_data, "UNALIGNED READ data zero")
389
390 // --- constrained-random traffic (run with +SEED=n +RANDN=count)
391 if (!$value$plusargs("SEED=%d", rseed)) rseed = 1;
392 if (!$value$plusargs("RANDN=%d", rnd_cnt)) rnd_cnt = 0;
393 rng = rseed ^ 32'hA5A5_1234;
394 for (rnd_n = 0; rnd_n < 8; rnd_n = rnd_n + 1) rng = xorshift(rng);
395 for (rnd_n = 0; rnd_n < rnd_cnt; rnd_n = rnd_n + 1) begin
396     rng = xorshift(rng); dut.ready_delay = rng[17:16];
397     rng = xorshift(rng); master.b_delay = rng[17:16];
398     rng = xorshift(rng); master.r_delay = rng[17:16];
399     rng = xorshift(rng); rnd_sel = rng[18:16];
400     case (rnd_sel)
401         0: rnd_addr = 32'h0000_0000;
402         1: rnd_addr = 32'h0000_0004;
403         2: rnd_addr = 32'h0000_0008;
404         3: rnd_addr = 32'h0000_000C;
405         4: rnd_addr = 32'h0000_0010;
406         5: rnd_addr = 32'h0000_0001;
407         6: rnd_addr = 32'h1000_0004;
408         default: rnd_addr = 32'h0000_0040;
409     endcase
410     rng = xorshift(rng); rnd_data = rng;
411     rng = xorshift(rng); rnd_strb = rng[19:16];
412     rng = xorshift(rng);
413     if (rng[16]) master.write(rnd_addr, rnd_data, rnd_strb);
414     else master.read (rnd_addr, read_data);
415 end
416 dut.ready_delay = 0; master.b_delay = 0; master.r_delay = 0;
417
418 $display("[TEST] EXTRA TESTS: %0d passed, %0d failed", ext_pass, ext_fail);
419
420 // END TEST
421
422 $display("");
423 $display("=====");
424 $display("          AXI4-LITE BASIC TEST COMPLETE");
425 $display("=====");
426 $display("");
427 #50;
428 $finish;
429 end
430
431 // WAVEFORM
432

```

```

433     initial begin
434         $dumpfile("sim/waves/axi4_lite.vcd");
435         $dumpvars(0, tb_top);
436     end
437 `ifdef ASSERTIONS
438     axi4_lite_sva #(
439         .ADDR_WIDTH(ADDR_WIDTH),
440         .DATA_WIDTH(DATA_WIDTH)
441     ) sva (
442         .aclk(aclk), .aresetn(aresetn),
443         .awaddr(awaddr), .awvalid(awvalid), .awready(awready),
444         .wdata(wdata), .wstrb(wstrb), .wvalid(wvalid), .wready(wready),
445         .bresp(bresp), .bvalid(bvalid), .bready(bready),
446         .araddr(araddr), .arvalid(arvalid), .arready(arready),
447         .rdata(rdata), .rresp(rresp), .rvalid(rvalid), .rready(rready)
448     );
449 `endif
450     // Watchdog: stop the simulation if a test hangs (e.g. a missing response)
451     initial begin
452         #500000;
453         $display("[TB] WATCHDOG TIMEOUT: simulation did not finish");
454         $finish;
455     end
456 endmodule

```

scripts/build.sh

3 lines

```
1 #!/bin/bash
2 cd "$(dirname "$0")/.." || exit 1
3 iverilog -g2012 -o sim/axi4_lite.out rtl/axi4_lite_slave.sv tb/master/axi4_lite_master.sv
  + tb/monitor/axi4_lite_monitor.sv tb/scoreboard/axi4_lite_scoreboard.sv
  + tb/coverage/axi4_lite_coverage.sv tb/tb_top.sv && vvp sim/axi4_lite.out
```

scripts/vbuild.sh

7 lines

```
1 #!/bin/bash
2 cd "$(dirname "$0")/.." || exit 1
3 verilator --binary --timing --assert -Wno-fatal -DASSERTIONS --top-module tb_top -Mdir sim/vl_obj -o
  + vtb \
4   rtl/axi4_lite_slave.sv tb/master/axi4_lite_master.sv tb/monitor/axi4_lite_monitor.sv \
5   tb/scoreboard/axi4_lite_scoreboard.sv \
6   tb/coverage/axi4_lite_coverage.sv tb/assertions/axi4_lite_sva.sv tb/tb_top.sv 2>&1 | grep -E
  + "%Error" | head -20
7 timeout 30 stdbuf -o0 ./sim/vl_obj/vtb 2>&1
```

scripts/mutate.sh

40 lines

```
1  #!/bin/bash
2  # Usage: scripts/mutate.sh <MUTANT_NAME>
3  cd "$(dirname "$0")/.." || exit 1
4  M=$1
5  D=/tmp/mut_`$M`
6  rm -rf "$D"; mkdir -p "$D"
7  cp rtl/axi4_lite_slave.sv "$D/slave.sv"
8  case "$M" in
9      MISSING_B)
10         sed -i "s/s_axi_bvalid <= 1'b1;/s_axi_bvalid <= 1'b0;/" "$D/slave.sv" ;;
11     MISSING_R)
12         sed -i "s/s_axi_rvalid <= 1'b1;/s_axi_rvalid <= 1'b0;/" "$D/slave.sv" ;;
13     BVALID_DROP)
14         sed -i "s/if (s_axi_bvalid && s_axi_bready) begin/if (s_axi_bvalid) begin/" "$D/slave.sv" ;;
15     RDATA_UNSTABLE)
16         sed -i "s/if (s_axi_rvalid && s_axi_rready) begin/if (s_axi_rvalid && !s_axi_rready)
+ s_axi_rdata <= ~s_axi_rdata;\n          if (s_axi_rvalid && s_axi_rready) begin/" "$D/slave.sv"
+ ;;
17     ILLEGAL_BRESP)
18         sed -i "/s_axi_bresp <= RESP_OKAY;/s/RESP_OKAY/2'b01/" "$D/slave.sv" ;;
19     ILLEGAL_RRESP)
20         sed -i "/s_axi_rresp <= RESP_OKAY;/s/RESP_OKAY/2'b01/" "$D/slave.sv" ;;
21     STRB_IGNORED)
22         sed -i "s/if (wstrb_reg[[0-3]\])/if (1'b1)/" "$D/slave.sv" ;;
23     ADDR_ALIAS)
24         sed -i "s/case ((\[A-Za-z_\]*\)\[31:4\] == 28'd0 && \[1:0\] == 2'b00) ? \[1:2\] : 4'hF)/case
+ (\[1:2\])/ " "$D/slave.sv" ;;
25     *) echo "unknown mutant: $M"; exit 1 ;;
26 esac
27 if cmp -s rtl/axi4_lite_slave.sv "$D/slave.sv"; then
28     echo "=== MUTANT: $M === MUTATION DID NOT APPLY (pattern not found)"; exit 1
29 fi
30 verilator --binary --timing --assert -Wno-fatal -DASSERTIONS --top-module tb_top -Mdir "$D/obj" -o
+ vtb \
31     "$D/slave.sv" tb/master/axi4_lite_master.sv tb/monitor/axi4_lite_monitor.sv \
32     tb/scoreboard/axi4_lite_scoreboard.sv \
33     tb/coverage/axi4_lite_coverage.sv tb/assertions/axi4_lite_sva.sv tb/tb_top.sv 2>&1 | grep "%Error"
+ | head -5
34 timeout 120 stdbuf -o0 "$D/obj/vtb" $MUT_ARGS > "$D/run.log" 2>&1
35 echo "=== MUTANT: $M ==="
36 echo "SVA rules that fired:"
37 grep -o "\[SVA_FAIL\] \[A-Z0-9_\]*" "$D/run.log" | sort | uniq -c | sed 's/^/ /'
38 first=$(grep -ml "\[SVA_FAIL\]" "$D/run.log")
39 echo "First SVA failure: ${first:-none}"
40 grep -E "SCOREBOARD RESULT|EXTRA TESTS|WATCHDOG" "$D/run.log" | sed 's/^/ /'
```

scripts/faults.sh

14 lines

```
1 #!/bin/bash
2 cd "${dirname "$0"}/.." || exit 1
3 mkdir -p sim/results
4 names=(x AWVALID_DROP AWADDR_CHANGE WVALID_DROP WDATA_CHANGE WSTRB_CHANGE ARVALID_DROP ARADDR_CHANGE)
5 for n in 1 2 3 4 5 6 7; do
6     timeout 60 stdbuf -o0 sim/vl_obj/vtb +FAULT=$n > /tmp/f_${n}.log 2>&1
7     inj=$(grep -m1 "FAULT_INJECTED" /tmp/f_${n}.log | grep -o "time [0-9]*" | cut -d' ' -f2)
8     line=$(grep -m1 "\[SVA_FAIL\]" /tmp/f_${n}.log)
9     rule=$(echo "$line" | awk '{print $2}')
10    t=$(echo "$line" | awk '{print $5}')
11    sb=$(grep "SCOREBOARD RESULT" /tmp/f_${n}.log | awk '{print $NF}')
12    if [ -n "$inj" ] && [ -n "$t" ]; then lat=$(( (t - inj) / 1000 )); else lat="n/a"; fi
13    echo "${names[$n]}: injected=${inj:-none} rule=${rule:-none} fail_t=${t:-none} latency_ns=$lat
+ scoreboard=${sb:-none}"
14 done | tee sim/results/faults.txt
```

scripts/random.sh

11 lines

```
1 #!/bin/bash
2 cd "$(dirname "$0")/.." || exit 1
3 mkdir -p sim/results
4 for s in 1 2 3 4 5 6 7 8 9 10; do
5     timeout 120 stdbuf -o0 sim/vl_obj/vtb +SEED=$s +RANDN=${RANDN:-100} > /tmp/r_$$s.log 2>&1
6     chk=$(grep -m1 "CHECKS =" /tmp/r_$$s.log | sed 's/ */ /g')
7     sb=$(grep "SCOREBOARD RESULT" /tmp/r_$$s.log | awk '{print $NF}')
8     sva=$(grep "SVA FAILS" /tmp/r_$$s.log | awk '{print $NF}')
9     tf=$(grep "EXTRA TESTS" /tmp/r_$$s.log | awk '{print $6}')
10    echo "seed=$s $chk scoreboard=${sb:-none} sva_fails=${sva:-none} tests_failed=${tf:-none}"
11 done | tee sim/results/random.txt
```

scripts/regress.sh

38 lines

```
1 #!/bin/bash
2 cd "$(dirname "$0")/.." || exit 1
3 mkdir -p sim/results
4 OUT=sim/results/regression_summary.txt
5 : > $OUT
6 echo "[1/4] Clean run, Icarus" >&2
7 {
8 echo "== CLEAN RUN (Icarus) =="
9 ./scripts/build.sh | grep -E "EXTRA TESTS|CHECKS|SCOREBOARD RESULT|COVERED BINS|COVERAGE  "
10 } >> $OUT
11 echo "[2/4] Clean run, Verilator with assertions" >&2
12 {
13 echo "== CLEAN RUN (Verilator + SVA) =="
14 ./scripts/vbuild.sh 2>&1 | grep -E "EXTRA TESTS|CHECKS|SCOREBOARD RESULT|SVA FAILS|SANITY"
15 } >> $OUT
16 echo "[3/4] RTL mutants (about 10-15 min, please wait)" >&2
17 : > sim/results/mutants.txt
18 for m in MISSING_B MISSING_R BVALID_DROP RDATA_UNSTABLE ILLEGAL_BRESP ILLEGAL_RRESP STRB_IGNORED
19 + ADDR_ALIAS; do
20     echo "    $m" >&2
21     ./scripts/mutate.sh $m >> sim/results/mutants.txt
22 done
23 {
24 echo "== RTL MUTANTS =="
25 awk '
26 function flush() { if (name != "") printf "%-16s SVA_caught=%-3s scoreboard=%-4s tests_failed=%s\n",
27 + name, sva, sb, tf }
28 /=== MUTANT:/ { flush(); name=$3; sva="NO"; sb="n/a"; tf="n/a" }
29 /First SVA failure:/ { if ($0 !~ /none/) sva="YES" }
30 /SCOREBOARD RESULT/ { sb=$NF }
31 /EXTRA TESTS/ { tf=$6 }
32 END { flush() }
33 ' sim/results/mutants.txt
34 } >> $OUT
35 echo "[4/4] Master-side faults" >&2
36 {
37 echo "== MASTER-SIDE FAULTS =="
38 ./scripts/faults.sh
39 } >> $OUT
40 cat $OUT
```